Integration and Performance Analysis of OPC UA and MQTT for Decentralized Industrial Control Between PLC and ROS 2 Environments

submitted as a requirement for the degree of

Master of Engineering and Sustainable Technology Management-
Industry 4.0: Automation, Robotics and 3D Manufacturing

Berlin, Germany, Date of Submission

by

Rohitsurya Sundara Rajan

B.E Mechanical Engineering, Cambridge Institute of Technology

Primary Thesis Supervisor:	Prof. Dr. Osvaldo Romero
Associate Thesis Supervisor:	Prof. Saiful Islam

written in conjunction with micropsi industries GmbH, Berlin, Germany

ACKNOWLEDGEMENTS

First and foremost, I thank God and the life force for placing me on such an incredible journey and for being my constant companion along the way. I am grateful for the guidance, strength, and opportunities that have shaped my path and allowed me to reach this milestone.

I wish to express my deepest gratitude to my parents, **Mr. Sundara Rajan** and **Mrs. Indira**, two remarkable individuals whose only dream has been to see my brother and me succeed in life. Their unwavering love, sacrifices, and encouragement have been the foundation of every step I have taken. I am equally thankful to my brother, **Mr. Sudarshan**, for his resilience and steadfast support as both a guide and a companion throughout my life.

To the love of my life, **Ms. Haritha**, I extend my heartfelt thanks for her patience, understanding, and constant belief in me as I pursued my dreams. Her presence has been a source of motivation during the most challenging times.

I would like to sincerely thank **Mr. Robert John**, Senior Automation Engineer at Micropsi Industries, for being an exceptional mentor in the field of robotics and automation. His guidance in understanding communication protocols, their implementation, and his invaluable support throughout this thesis have been instrumental to my success.

My gratitude also goes to **Mr. Maximilian Mutschler**, Vice President of Sales at Micropsi Industries. Under his leadership, I had the privilege to work on projects that enhanced my technical capabilities. His vision, dedication, and ability to inspire his team have left a lasting impact on me. I am proud to have worked under his guidance and to have been part of such an innovative and forward-thinking team.

I also wish to thank **Micropsi Industries GmbH** for giving me the opportunity to be part of their journey. My time with the company not only helped me grow professionally but also shaped my perspective on technology and its applications in the real world.

Finally, I acknowledge that during the course of this thesis, **ChatGPT (OpenAI GPT-4)** was used selectively to assist in debugging and troubleshooting code, particularly during the development of custom Communication Protocol and simulation plugins. All final code implementations were written, verified, and adapted independently by the author based on a clear understanding of the system requirements.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS ii
TABLE OF CONTENTS iii
LIST OF FIGURES v
LIST OF TABLES vii
LIST OF EQUATIONS viii
LIST OF ABBREVIATIONS ix
1. INTRODUCTION 1
4.1. Subject Relevance 1
4.1. Own Interest/Motivation 3
4.1. Research Gap 4
4.1. Objectives and Goals 5
4.1. Research Questions 7
4.1. Scope & Limitations 8
2. LITERATURE REVIEW & THEORETICAL FRAMEWORK 9
4.1. Conceptual Foundations in Industrial Communication 9
4.1. Protocol Specific Reviews 10
4.1. Theoretical Framework 11
4.1. Research Strategies: 12
3. METHODOLOGY 13
4.1. Roadmap 13
4.1. Environment & Toolchain 14
4.1. Digital Twin Simulation (ROS2 + Gazebo) 17
2.3.1 Model Design 18
2.3.2 Model Plugins 20
This subsection describes the custom Gazebo model plugin that connects the simulated box to ROS 2, applies incoming target poses deterministically inside the physics loop, and exposes lightweight diagnostics for timing analysis. 20
2.3.3 Box Control logic: 23
2.2. Communication and Time Synchronization: 27
2.4.1 Communication Protocol 27
2.4.2 Time Synchronization: 32
2.3. How it works? 34
2.5.1 Core simulation (common to OPC UA and MQTT): 34
3.5.2 How OPC UA drives and reads the simulation 36
3.5.3 How MQTT drives and reads the simulation 37
4. FINDINGS 38
4.1. Data Loss 39
4.1.1 Relevance to This Research: 39
4.1.2 Measurement Method 39
4.1.3 Results: 40
4.1.4 Implications for Further Analysis 41
4.2. Trigger-to-ROS Latency 41
4.2.1 Relevance to This Research: 41
4.2.2 Measurement Methodology: 42
4.3.3 Results: 43
4.2.4 Graphical Results: 45
4.3. Pose-to-Feedback Latency 50
4.3.1 Relevance to This Research: 50
4.3.2 Measurement Methodology: 51

4.3.3 Results: 52
4.3.4 Graphical Results: 54
4.4. Jitter Analysis: 59
4.4.1 Relevance to This Research: 59
4.4.2 Measurement Methodology: 60
4.5. Throughput Analysis: 64
4.4.2 Relevance to This Research: 64
4.4.2 Measurement Methodology: 65
5. DISCUSSION OF FINDINGS 72
6. CONCLUSION 75
APPENDIX I ..... - 1 -
ALPHABETICAL LIST OF REFERENCES ..... - 2 -
CHRONOLOGICAL LIST OF REFERENCES ..... - 6 -
REFERENCE LIST BY BIBLIOGRAPHY TYPE ..... - 10 -
AFFIDAVIT ..... - 1 -

LIST OF FIGURES

Figure 1: Hardware specifications for simulation and control hosts – Laptop..... 15

Figure 2: Hardware specifications for simulation and control hosts – Virtual Box..... 15

Figure 3: Gazebo Simulation of the "factory.world" 18

Figure 4: Folder tree – factory_simulation. 22

Figure 5: Folder tree – Box_Pose_Monitor..... 26

Figure 6: Folder tree – Box_Teleport_Node. 26

Figure 7: OPCUA – Working Principle. 27

Figure 8: MQTT – Working Principle..... 30

Figure 9: NTP Time server Monitor..... 33

Figure 10: RQT Graph – Nodes in OPCUA (Command Leg) 34

Figure 11: RQT Graph – Nodes in MQTT (Command Leg) 34

Figure 12: RQT Graph – Nodes in OPCUA (Feedback Leg) 35

Figure 13: RQT Graph – Nodes in MQTT (Feedback Leg)..... 35

Figure 14: RQT Graph – OPCUA 36

Figure 15: RQT Graph – MQTT 37

Figure 16: Latency Over Time – MQTT 45

Figure 17: Latency Over Time – OPCUA..... 46

Figure 18: Latency Distribution – MQTT 47

Figure 19: Latency Distribution – OPCUA..... 48

Figure 20: Latency Distribution – MQTT 54

Figure 21: Latency Distribution – OPCUA..... 55

Figure 22: Latency per Pose – MQTT..... 56

Figure 23: Latency per Pose – OPCUA 57

Figure 24: Jitter per Pose – MQTT 61

Figure 25: Jitter per Pose – OPCUA 62

Figure 26: Throughput per run – OPCUA 67

Figure 27: Throughput per run – MQTT 67

Figure 28: Throughput Differences per run – OPCUA 69

Figure 29: Throughput Differences per run – MQTT 69

LIST OF TABLES

Table 1: Data Loss Summary for OPC UA and MQTT 40

Table 2: Trigger Latency for MQTT 43

Table 3: Trigger Latency for OPCUA 44

Table 4: Trigger Latency Summary for OPCUA vs MQTT 44

Table 5: Pose to Feedback Summary for MQTT 52

Table 6: Pose to Feedback Summary for OPCUA 53

Table 7: Jitter Summary for MQTT 61

Table 8: Jitter Summary for OPCUA 62

Table 9: Throughput Summary for MQTT 66

Table 10: Throughput Summary for OPCUA 66

LIST OF EQUATIONS

Equation 1: Latency Calculation 43

Equation 2: Interval Calculation for Jitter 60

Equation 3: Interval Calculation for Jitter 60

Equation 4: ROS throughput Calculation 65

Equation 5: Windows throughput Calculation 65

LIST OF ABBREVIATIONS

Acronym	Explanation
IIoT	Industrial Internet of Things
IoT	Internet of Things
JSON	JavaScript Object Notation
PLC	Programmable Logic Controller
QoS	Quality of Service
ROI	Return on Investment
ROS 2	Robot Operating System 2
SCADA	Supervisory Control and Data Acquisition
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UA	Unified Architecture
URL	Uniform Resource Locator

1. INTRODUCTION

Intelligent automation, industrial communication protocols and the integration of information technology and physical systems are transforming manufacturing operations in the era of Industry 4.0. Modern smart factories demand not only efficient automation but also require dependable and smooth communication between digital control systems and actual machines. The necessity to assess and put into practical communication protocols that can deliver predictable performance, low latency, and resilience under dynamic operating situations is at the heart of this change.

In the framework of decentralized control in a manufacturing simulation, this thesis investigates and assesses the communication capabilities of OPC UA (Unified Architecture) and MQTT (Message Queuing Telemetry Transport) protocols. The goal is to evaluate how well they engage for sending time-sensitive data between a robotic simulation framework built on ROS 2 and Gazebo and a Programmable Logic Controller (PLC) system. The study focuses on important performance measures that are essential for real-time industrial applications, including latency, jitter, throughput, and data loss

Using ROS 2 Humble and Gazebo, a simulation environment was created in which a box travels down a conveyor system to mimic actual manufacturing activity. Pose updates and real-time position feedback monitoring are handled by a specially created robotic system that includes ROS 2 nodes such as box_teleport_node and box_pose_monitor. The system is designed to interface with an external PLC-like control layer, enabling command and feedback exchange via OPC UA and MQTT protocols.

4.1. Subject Relevance

The combination of real-time automation systems and cutting-edge communication technology is revolutionizing factory operations in the context of Industry 4.0 and the Industrial Internet of Things (IIoT).

Industrial automation has traditionally been built on conventional centralized control techniques, such as central master-slave communication models, SCADA-based designs, or hardwired PLC systems. These systems frequently have rigid setups, single points of

failure, and restricted scalability since they rely on a central controller that processes all data and sends commands to linked devices. Nonetheless, decentralized, distributed architectures are becoming increasingly common among modern industrial systems, requiring dependable, scalable, and efficient communication between sensors, controllers, robotic systems, and gadgets.

Industrial communication protocols like OPC Unified Architecture (OPC UA) and MQTT (Message Queuing Telemetry Transport) have become strong candidates to satisfy these shifting requirements since both offers the flexibility, interoperability, and robustness needed for dynamic smart factory settings.

Because of its support for intermediary layering and increasing use in industry research, ROS 2 (Robot Operating System) is a solid platform for creating and assessing robotic systems. ROS 2 enables the creation of intricate and lifelike cyber-physical ecosystems to test manufacturing workflows prior to deployment when paired with simulation tools such as Gazebo. For smart manufacturing infrastructure design and protocol selection to be well-informed, communication protocol's performance in these simulated environments must be assessed.

The key words associated with this work include:

- **OPC UA:** A service-oriented, platform-independent protocol for safe and dependable data transfer in industrial systems.
- **MQTT:** It is a popular lightweight publish-subscribe messaging protocol for effective telemetry data transfer in IIoT contexts.
- **ROS 2:** A modular middleware for robots that facilitates DDS-based communication and real-time, scalable system design.
- **Gazebo:** Robots and mechanical systems can be modelled and tested in virtual settings using Gazebo, a physics-based simulation tool.

In recent research, industrial communication protocols like OPC UA and MQTT have been assessed in lab settings, frequently focusing on particular metrics like throughput, packet loss, and latency. (Daniel Silva, 2021), for example, compared MQTT, CoAP, and OPC UA in their paper "A Performance Analysis of Internet of Things Networking Protocols: Evaluating MQTT, CoAP, OPC UA" across various network configurations and discovered notable performance variations based on use case and architecture design. Metrics including latency, packet loss, and throughput have been the subject of another recent laboratory evaluations of industrial communication protocols like OPC-UA and MQTT. For example, the study "OPC UA versus ROS, DDS, and MQTT Performance Evaluation of Industry 4.0 Protocols" (Stefan Profanter, 2019) performed a thorough benchmarking comparison of OPC UA, ROS, DDS, and MQTT in an Industry 4.0 context, measuring protocol overhead and round-trip times under various CPU and network load conditions.

Although this study provides insightful information at the protocol level, it doesn't examine end-to-end integration in robotic systems, especially in a simulation environment based on ROS 2.

The performance of OPC-UA and MQTT when closely linked with ROS 2 and Gazebo in a comprehensive system remains a crucial gap, despite ROS 2's growing use for decentralized control in smart manufacturing. By creating a simulation-based testbed with ROS 2 Humble and Gazebo and methodically contrasting OPC-UA with MQTT using predetermined communication performance parameters, this thesis closes that gap.

4.1. Own Interest/Motivation

My academic and professional experiences with robotic automation, system integration, and data-driven process improvement served as inspiration for this effort. My work setting up industrial robots, performing Proof-of-Concepts, and programming automation workflows made me realize how important communication protocols are to control and decision-making in real time. This thesis gave me a special chance to integrate my passions for simulation, network communication, and robotics to produce a comprehensive, testable factory model.

4.1. Research Gap

Despite the fact that OPC UA and MQTT have been thoroughly examined in relation to industrial automation, the majority of current research concentrates on assessing each protocol separately and apart from robotics middleware like ROS 2. MQTT is prized for its lightweight and bandwidth-efficient telemetry, especially in IoT and edge-to-cloud connection, whereas OPC UA is well known for its organized and secure information modelling, which makes it appropriate for sophisticated industrial data sharing.

In ROS 2-controlled robotic systems, where the communication layer is essential to establishing consistent control loops and low-latency feedback, however, relatively few studies have carefully evaluated these two protocols. There is especially little research on how well they work as real-time communication links between PLC (or PLC-equivalent) systems and robotic simulators in dynamic, factory-like settings.

The absence of assessments that include real-time feedback loops, constant pose monitoring, and live simulation data streams all crucial for precise industrial performance evaluations is another gap. Prior research frequently misses the timing behavior and synchronization issues that arise in actual production circumstances when this integration is lacking.

Furthermore, while developing ROS 2-based digital twin architectures, industrial engineers and system integrators are not given much assistance in the literature on protocol selection. Consequently, rather than using evidence-based performance measurements, decision-making frequently depends on anecdotal experience or vendor suggestions.

These weaknesses are directly addressed in this thesis by:

1. Using Gazebo and ROS 2 to construct a replicable digital twin environment that resembles a streamlined manufacturing cell.
2. Connecting the control logic and the robotic simulation using both OPC UA and MQTT as a communication layers.
3. Using the same test settings to benchmark important performance indicators, such as throughput, packet loss, jitter, and latency.

This study offers precise, data-driven insights into which protocol is more appropriate for ROS 2-based industrial systems by evaluating findings in a controlled and repeatable experimental framework. This information will help guide future integration choices in automation and smart manufacturing initiatives.

4.1. Objectives and Goals

This thesis's main goal is to assess the suitability and performance of two most popular industrial communication protocols, Message Queuing Telemetry Transport (MQTT) and OPC Unified Architecture (OPC UA), within the framework of a robotic simulation environment controlled by ROS 2. This work focuses on a system-level implementation that creates a digital twin environment to simulate real-world industrial communication between a PLC and robotic cell, rather than just theoretical or isolated benchmarking.

This is accomplished by utilizing ROS 2 Humble and Gazebo to create a realistic manufacturing simulation in which a box travels along a conveyor system. Custom ROS 2 nodes that manage position changes and real-time feedback are part of the robotic system. After that, an external layer is interfaced with this simulation environment, enabling communication via MQTT and OPC UA.

The specific goals of this thesis are to:

- Create a replicable digital twin simulation architecture that connects an external control source acting as a PLC-equivalent layer to a robotic system controlled by a ROS 2. Because licensed PLC software and industrial controller hardware are unavailable, external scripts and logic blocks are used to mimic PLC functionality. This method preserves flexibility for protocol switching and parameter changes while allowing the testing of command execution and feedback loops in a setting that closely resembles an actual manufacturing setup.

- Create and manage pose-based motion and feedback systems in Gazebo by utilizing the box_pose_monitor and box_teleport_node, two unique ROS 2 nodes.
  In order to provide realistic test circumstances for protocol latency and reliability measurement, these nodes allow the system to send motion commands, update the box's position in the simulation, and continually monitor the actual pose.

- Include both MQTT and OPC UA as equivalent communication protocols between the ROS 2 simulation and the PLC-equivalent control layer.
  In order to explicitly attribute performance differences to protocol behaviour rather than modifications to system architecture, each protocol will be set up to handle the same tasks: transmitting control triggers and receiving continuous pose data.

- Benchmark critical communication performance metrics under controlled and repeatable test scenarios, including:

  1. **Latency** – both one-way (command to execution) and round-trip (command to acknowledgment).
  2. **Packet Loss Rate** – the percentage of expected messages that fail to arrive.
  3. **Throughput** – the rate of successful message delivery per unit time.
  4. **Jitter** – variation in message arrival times, which affects stability in real-time control.

- Examine how the protocol behaves under constant operating loads to evaluate its responsiveness, stability, and capacity for long-term, continuous data exchange. This entails monitoring how each protocol continues to function as the number of messages increases and seeing any trends in instability or degradation.

- Determine which protocol is more appropriate for industrial digital twin setups based on ROS 2, such the one created in this thesis.
  Based on the performance data gathered, a comparison of OPC UA and MQTT is conducted in order to determine what is best between structured data processing, communication effectiveness, integration convenience, and real-time responsiveness. When deciding between these protocols for comparable robotic and industrial automation scenarios, the result seeks to offer engineers, system integrators, and researchers practical insights.

4.1. Research Questions

In a simulated smart factory setting, this thesis explores the integration of robotic systems with industrial communication protocols. In addition to assessing communication performance, the goal is to investigate the architectural consequences of using middleware protocols to connect robotic actuation and control logic.

The work is influenced by the following research questions:

1. **In a digital twin setting, how can ROS 2 be successfully combined with PLC-equivalent control logic to provide decentralized control of robotic cells?**
   The architectural design and real-world implementation of a ROS 2-based system that mimics PLC capability by receiving external control inputs and reacting with real-time actuation and feedback are examined in this subject.

2. **What are OPC UA and MQTT's latency and packet loss characteristics in a robotic simulation based on ROS 2 and Gazebo?**
   With particular attention to one-way and round-trip delay, jitter, message loss, and throughput, this inquiry centres on the quantitative assessment of both protocols under controlled test conditions.

3. **What might be the impact on the use of MQTT and OPC UA have on industrial communication systems?**
   In situations where two protocols are implemented together or alternately in the same automation architecture, this subject discusses protocol coexistence, possible bottlenecks, synchronization problems, and design trade-offs.

4.1. Scope & Limitations

The performance of the industrial communication protocols OPC UA and MQTT is assessed in this thesis using ROS 2-controlled robotic simulations. Using ROS 2 Humble and Gazebo, the project involves creating a digital twin environment in which a virtual box travels on a conveyor system and pose data is shared with an external control layer.

Scope:

- The simulation environment simulates a simple robotic cell common in smart manufacturing settings, with posture updates handled using ROS 2 nodes
- OPC UA and MQTT are used in the same design, but independently for protocol assessment.
- Key communication performance parameters are benchmarked, including latency, jitter, packet loss, and throughput.
- Custom logic is used to externally reproduce the control logic that a PLC would normally manage, eliminating the need for commercial PLC hardware or software.
- The system focuses customization, real-time feedback, and scalability in order to replicate decentralized industrial control.

Limitations:

- Because tools availability restrictions, the system simulates PLC capabilities externally rather than using real PLC hardware or software.
- Instead of using cloud, wide area network, or industrial-grade fieldbus networks, the performance is evaluated over a local area network (LAN).
- This study does not assess security features like encryption or OPC UA and MQTT authentication methods.
- Since the results are based on simulation, they might not match actual industrial installations where communication behaviour is impacted by PLC firmware, hardware latency, or noise.
- Only single-point communication flows (trigger and feedback) are evaluated in this study. Protocols used in actual industrial systems may include timers, numerous publishers/subscribers, or intermediary broker/server characteristics that impact performance results. We didn't look at such multi-point interactions or sophisticated protocol features here.

2. LITERATURE REVIEW & THEORETICAL FRAMEWORK

4.1. Conceptual Foundations in Industrial Communication

- **Compatibility in Systems for Industry 4.0:**
  Modern industrial automation systems, which require smooth communication across disparate equipment and software platforms, are fundamentally dependent on compatibility. In the Paper (Isaías González, 2019) showcase the evolution of Open Platform Communications (OPC) to solve compatibility issues in modern manufacturing settings, emphasizing the move to OPC UA's service-oriented architecture for platform-independent and scalable integration. Similarly from (Abdulkadir Karaagac, 2019) talk about how OPC UA can be combined with other IIoT protocols, such Lightweight M2M (LwM2M), to produce hybrid systems that are semantically rich and lightweight.

- **Digital Twin Simulation in Manufacturing**
  The idea of "digital twins" is being increasingly utilized to imitate real-world industrial processes in order to monitor, validate, and optimize them in real time. (Matteo Pernoa, 2021) use Gazebo and ROS to simulate digital twins for industrial control, providing information on how simulation may help with defect prediction and latency-aware control design.

- **Real-Time Control and Communication Constraints**
  Timelines for industrial applications are frequently very tight. (Jaloudi, 2019) gives a comparative analysis of different protocols in terms of packet loss, determinism, and latency all of which are crucial in automated procedures that require quick turnaround times. Also, (Salvatore Cavalieri, 2013) provide a targeted evaluation of OPC UA's performance in terms of data throughput, determinism, and latency all of which are crucial in automation workflows that require quick turnaround times

4.1. Protocol Specific Reviews

- **OPC UA – Structured, Secure, and Scalable**
  Because of its platform freedom, security features, and information modelling capabilities, OPC UA has emerged as a prominent standard in industrial communication. (Aparajita Tripathy, 2022) demonstrate how OPC UA may be successfully incorporated into local cloud settings to enable high interoperability for robotic control via ROS.

  (Seung-Yong Lee, 2022) examine how OPC UA can be used in conjunction with PLCs to communicate data securely, highlighting how widely used it is in critical infrastructure.

  (Stefan Profanter, 2019) compare OPC UA to ROS, DDS, and MQTT, pointing out that while it has more overhead, it is better tailored for machine-to-machine communication in controlled industrial environments.

  (Marc Ladegourdie, 2022) prove that OPC UA is feasible for edge computing by maintaining respectable performance even on limited hardware platforms.

  (L. Freitas, 2024) proceed by comparing the performance of OPC UA implementations under various message loads and setups. Protocol tuning in digital twin settings, such as the one in this thesis, is informed by their research.

- **MQTT – Lightweight and IoT-Friendly**
  MQTT is perfect for telemetry, streaming sensor data, and edge-to-cloud communications since it is built for low-overhead, publish-subscribe communication. (Usmani, 2021) describe how MQTT is appropriate for Industrial IoT systems with random connectivity due to its asynchronous nature and low payload overhead.

  (Banno, 2023) analyse MQTT under varying traffic loads, demonstrating that although MQTT provides good throughput and latency performance, it may not function well in situations with a lot of traffic or time constraints. In robotic

systems, where steady feedback rates are necessary for seamless control, this is especially relevant.

(Cüneyt Bayılmıs, 2022) give a deeper examination of MQTT and other IIoT protocols, pointing out areas in which MQTT works well and others that could require structured alternatives like OPC UA.

- **ROS 2 Middleware – Bridging Robotics and Industrial Communication**
  A versatile framework that works well with industry standards is ROS 2. (Tobias Kronauer, 2021) analyses the latency of ROS 2 multi-node systems, identifying any bottlenecks in the middleware in real time.

  (Endre Erős, 2019) provide an architecture for collaborative automation based on ROS 2, demonstrating how ROS 2 can grow to accommodate complicated systems with decentralized control loops.

  (Martin Dahl, 2019) suggest Sequence Planner, a program that enhances ROS 2's planning and control features for scenarios involving intelligent automation.

4.1. Theoretical Framework

This thesis draws on the following theoretical principles:

- Cyber-Physical Systems (CPS): The simulation serves as a CPS, using ROS 2 and Gazebo to digitally replicate the physical process (conveyor and box).
- Decentralized Control Theory: The architecture makes use of distributed logic, triggering, and feedback routing using message-passing protocols (MQTT and OPC UA) in place of centralized SCADA/PLC installations.
- Queueing Theory: Using time-series data and models of queue delay, congestion, and throughput, latency, jitter, and packet loss are examined.
- Digital Twin Modelling: The simulation's temporal accuracy, feedback consistency, and synchronization reliability all crucial elements of a working digital twin in industrial automation contexts are evaluated in addition to its visual depiction.

4.1. Research Strategies:

A combination of back-tracing and systematic search strategies were used to guarantee a thorough and exacting literature evaluation. Academic databases like IEEE Xplore, ScienceDirect, SpringerLink, and MDPI were used to find primary materials. Keywords like "OPC UA," "MQTT," "ROS 2," "digital twin," "industrial communication protocols," and "smart manufacturing" were used.

To find peer-reviewed publications that discuss the functionality, integration, and interoperability of industrial communication protocols in robotic and cyber-physical systems, a methodical literature review technique was used. Priority was given to studies that were published between 2011 and 2024 in order to represent both recent developments and foundational work.

Backtracing was used to find relevant or earlier research that might not have come up in initial searches by looking through the bibliographies of important papers (e.g., (Stefan Profanter, 2019), (Cüneyt Bayılmıs, 2022)., (Aparajita Tripathy, 2022)). This made themes like digital twin validation, ROS 2 integration, and protocol benchmarking more consistent.

The theoretical framework and practical design of the digital twin simulation employed in this thesis were guaranteed to have a well-rounded and excellent basis thanks to this blended research approach.

3. METHODOLOGY

4.1. Roadmap

This chapter outlines the methodology utilized to address the identified research gap the lack of a systematic, end-to-end performance evaluation of OPC UA and MQTT in a digital twin environment constructed with ROS 2 and Gazebo, simulating a conveyor-based robotic process with PLC-equivalent control logic. In contrast to hardware-based industrial testbeds, which use real robots, conveyors, and PLCs, this study uses a fully virtualized method that simulates the same operating logic.

Even though previous research has independently assessed each protocol's strengths OPC UA for its structured and secure information modelling, and MQTT for its lightweight, efficient telemetry there is currently little guidance on which protocol is better suited for ROS 2-controlled industrial simulations that replicate real-time robotic operations, continuous pose tracking, and closed-loop feedback.

The main objective of this study is to create, put into practice, and assess a repeatable testbed that allows for a direct and equitable comparison of these two protocols in terms of sending feedback and control signals in real time between a ROS 2-controlled robotic simulation and a PLC-equivalent control layer. In order to offer evidence-based suggestions for protocol selection in comparable configurations, this study will benchmark key performance indicators latency, jitter, packet loss, and throughput under identical, controlled conditions.

The approach is divided into several essential parts:

1. **Simulation Environment Development:** A digital twin of a streamlined manufacturing process with a moving box and conveyor was made using ROS 2 Humble and Gazebo. Pose control, position monitoring, and feedback publishing are handled by custom ROS 2 nodes (box_teleport_node, box_pose_monitor).

2. **Implementation of PLC-Equivalent Control Logic:** External control logic was created to mimic PLC behaviour, processing feedback and sending motion orders, in place of real industrial controller hardware and software.

3. **Protocol Integration:** To ensure a fair performance comparison, OPC UA and MQTT were combined as interchangeable communication layers, each set up to carry out the same responsibilities.

4. **Network and Time Synchronization Setup:** To enable accurate latency measurements, network time synchronization was put in place, and a dedicated LAN was set up to reduce latency fluctuation and isolate test traffic.

5. **Experimental Design and Data Collection:** To collect statistically relevant performance data, each protocol underwent many test runs with the same set of parameters.

6. **Analysis and Comparative Evaluation:** To calculate the specified metrics, examine protocol behaviour under load, and determine which protocol best satisfies the performance requirements of ROS 2-based industrial applications, the collected data was processed.

The chapter begins by describing the hardware and software environment that was used. It then goes on to explain the simulation design and control logic in detail, the implementation and configuration of each protocol, the LAN, and the time synchronization setup. The experimental protocol, data analysis techniques, and a review of the method's advantages and disadvantages are addressed in the findings section.

4.1. Environment & Toolchain

This section details the whole hardware, software, and network configuration utilized throughout the study to guarantee that the experimental results are repeatable and verifiable by other researchers. Hardware, operating systems, middleware, simulation software, protocol stacks, and network configuration were all chosen to meet the needs of precise performance benchmarks and real-time, low-latency communication.

A two-machine LAN configuration was used for all experiments: one system served as the PLC-equivalent control client, and the other system ran the ROS 2 simulation and protocol servers. The details are given in the below images.

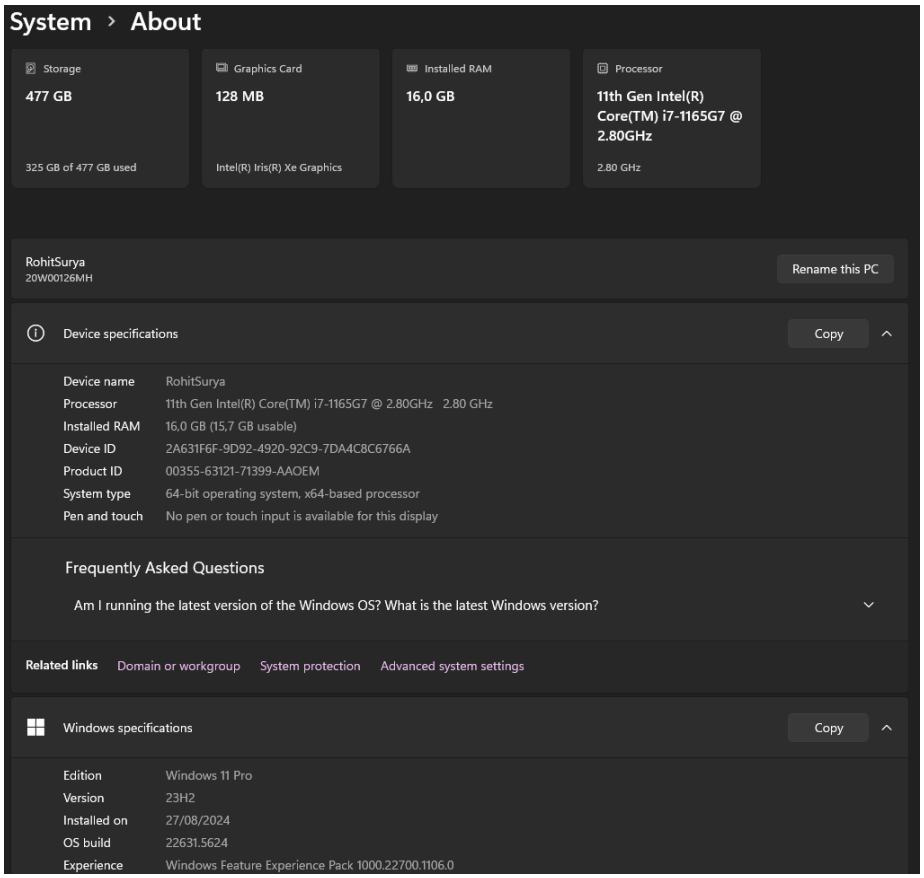

Figure 1: Hardware specifications for simulation and control hosts – Laptop

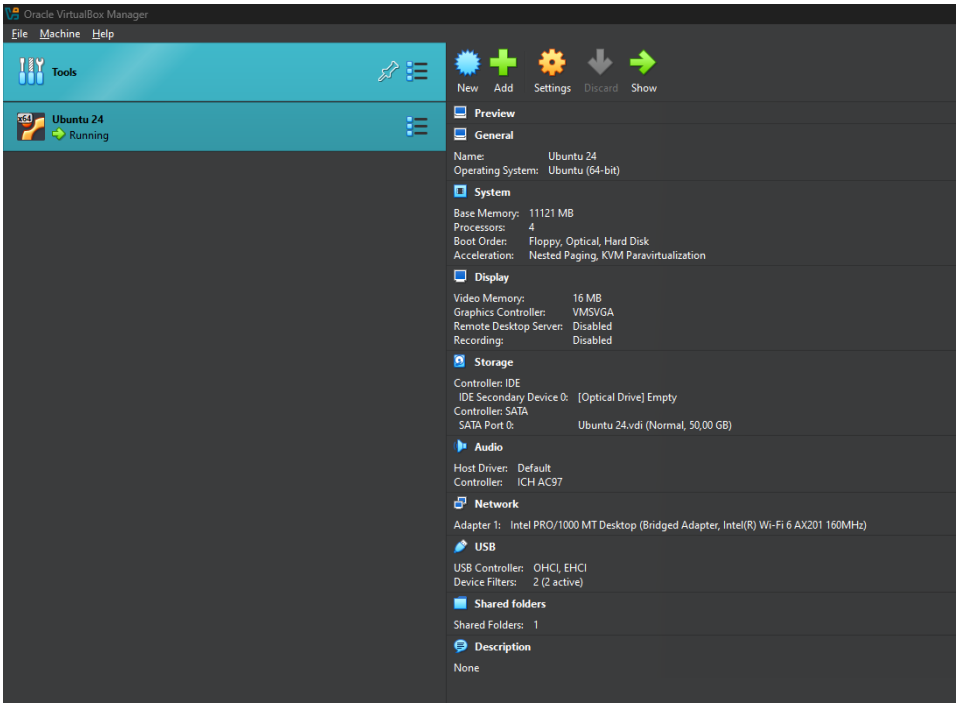

Figure 2: Hardware specifications for simulation and control hosts – Virtual Box

1. **Operating System:**
    - **ROS 2 Simulation Host:** Ubuntu 22.04.5 LTS
    - **PLC-Equivalent Host:** Windows 11 Pro (23H2 build) with latest patches

Both systems were kept in **standalone LAN mode** to prevent internet traffic from introducing variability.

2. **ROS 2 Middleware:**
    - **Distribution:** ROS 2 Humble Hawksbill
    - **Core Packages:** rclpy, geometry_msgs, std_msgs, gz-transport12, gz-sim7
    - **Gazebo Version:** Gazebo Sim 7.9.0 (Fortress series)
    - **Models Used:** Custom-designed SDF models for a conveyor system and a moving box.

3. **Programming Languages & Libraries:**
    - **Python:** 3.10.12
    - **ROS 2 Nodes:** Implemented in Python using rclpy.
    - **Libraries:**
        - paho-mqtt — MQTT client for broker communication.
        - freeopcua — OPC UA server/client for protocol integration.
        - numpy, pandas — data processing and metric computation.
        - matplotlib — plotting and visualization of performance metrics.

4. **Miscellaneous Tools:**
    - **MQTT Broker:** Mosquito
    - **Time Synchronization:** NTP Time Server

4.1. Digital Twin Simulation (ROS2 + Gazebo)

The main experimental platform for assessing OPC UA and MQTT's performance in an industrial setting with ROS 2 control is the digital twin created for this thesis. Digital twins are virtual representations of real-world industrial processes, such as conveyor-based robotic operations, that mimic data flows, control logic, and system reactions. Without using actual industrial hardware, the digital twin allows for accurate, repeatable, and isolated performance testing by simulating the process's physical dynamics as well as the communication interactions with an external controller.

Gazebo Sim (version 7.9.0, Fortress series) was used as the physics-based simulation engine and ROS 2 Humble Hawksbill as the middleware framework to create the simulation environment. ROS 2 was selected due to its ability to integrate several industrial communication protocols, its middleware abstraction using DDS, and its strong support for distributed, modular robotics systems. Gazebo was chosen because of its sophisticated physics engine, capacity to simulate realistic dynamics, sensor data, and actuator behaviour, and interoperability with ROS 2 via the gz-transport12 and gz-sim7 interfaces.

The research gap mentioned in Chapter 1 is immediately addressed by this strategy. In contrast to real testbeds, which require specialized hardware like industrial robots, conveyors, and programmable logic controllers (PLCs), the simulation enables controlled, repeatable trials with precisely manipulable variables. This is crucial for protocol benchmarking because it allows protocol performance to be isolated as the main variable by testing OPC UA and MQTT under the same operational conditions, such as the same command frequencies, message payloads, and network setups.

Additionally, live simulation data, continuous position monitoring, and real-time feedback loops features that are frequently absent from earlier protocol evaluations can be incorporated using the digital twin. These features preserve the ability to alter system parameters for experimental reasons while maintaining all the outcomes are reflective of actual industrial control scenarios.

2.3.1 Model Design

Using a combination of a custom world file (factory.world) and model files (model.sdf and model.config), the simulated manufacturing environment was built in Gazebo Sim (version 7.9.0, Fortress series) to depict a simplified conveyor system with a moveable box. With little visual and computational complexity, this design allows for the replication of key spatial relationships and physical interactions present in an actual industrial cell, allowing for reproducible experimentation.

i. World File – factory.world

The factory.world file serves as the top-level configuration for the simulation. It defines:

- Global settings such as gravity, physics solver parameters, and lighting.
- World frame origin and ground plane.
- Inclusion of the custom conveyor and box models at predefined coordinates.

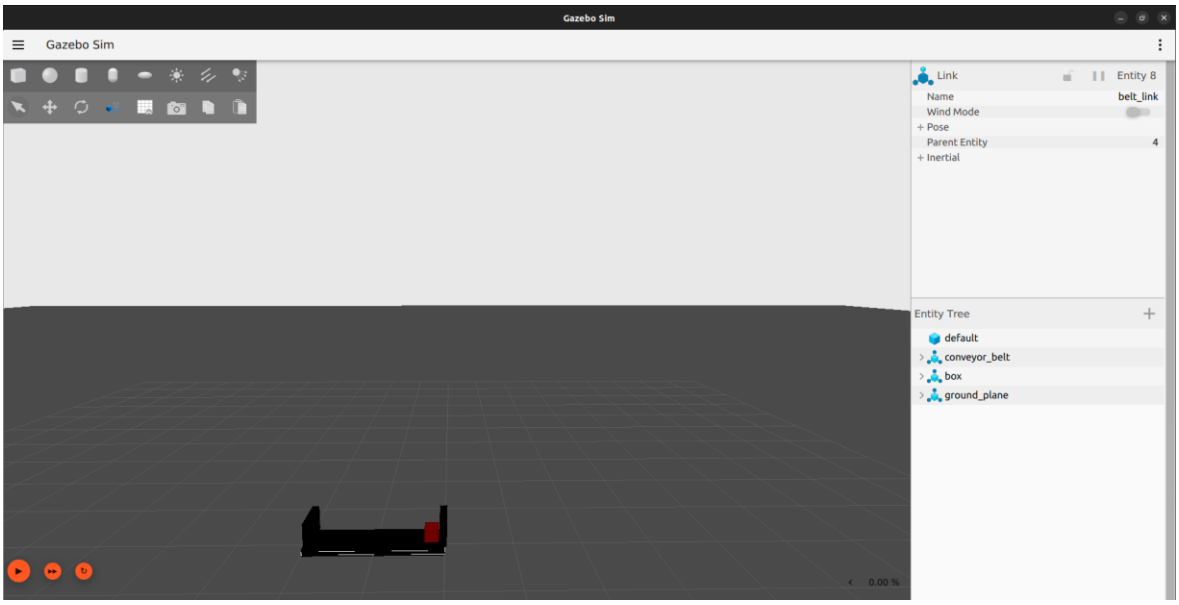

Figure 3: Gazebo Simulation of the "factory.world"

This file is loaded at the start of each simulation trial to ensure that the scene is initialized identically across all test runs.

ii. Box Model - model.sdf

The box is modelled as a simple cuboid primitive to represent a standard industrial workpiece. Key parameters defined in its SDF file include:

- **Initial pose:** Set relative to the world frame to ensure consistent starting coordinates across all trials.

The simplicity of the box model minimizes computational overhead, ensuring that variations in measured communication performance are not influenced by unnecessary physics complexity.

iii. Conveyor Model - model.sdf

The conveyor is represented as a static structure with:

- **Base frame**: Acts as a fixed reference for the simulation.
- **Collision surface:** Represents the belt area on which the box rests and moves.
- **Static property:** The conveyor model itself is static; belt motion is simulated through pose updates to the box rather than physical belt rotation, ensuring precise and repeatable movements.

This design ensures the conveyor serves as a stable environment for motion control experiments, removing physical belt irregularities as a confounding factor.

iv. Coordinate Frames

The simulation uses Gazebo's standard right-handed coordinate convention:

- **X-axis:** Forward direction along the conveyor belt.
- **Y-axis:** Lateral direction across the belt.
- **Z-axis:** Upward from the ground plane.

These frames provide a consistent spatial reference for both sending control commands and monitoring feedback.

v. Justification for Conveyor and Sliding Box Use:

To keep the simulation setup straightforward, computationally light, and reproducible, a conveyor with a sliding box was chosen. However, the information being collected through control commands being sent and feedback received is universal and can be used with other mobile or robotic systems.

The fundamental idea is the same whether the target is a robotic manipulator, an autonomous guided vehicle (AGV), or a turtlesim: a command message is sent to the system, and it responds with real-time feedback. This universally guarantees that the findings of this study's protocol performance analysis are applicable outside of the particular conveyor-box configuration.

vi. Appendix References

Only the primary model files directly related to the experimental activity are included in the appendix:

- Appendix 1.1: factory.world – The factory world.
- Appendix 1.2: model.sdf – Box model.
- Appendix 1.3: model.sdf – Conveyor model.

2.3.2 Model Plugins

This subsection describes the custom Gazebo model plugin that connects the simulated box to ROS 2, applies incoming target poses deterministically inside the physics loop, and exposes lightweight diagnostics for timing analysis.

i. BoxPosePublisher (feedback publisher)

What it is:

A minimal ROS 2 publisher used to drive the simulation by sending the current box poses. By knowing where the box is present, the change in position can be applied easily to move the box from its current position to the targeted position.

What it does:

- Creates a ROS 2 node and publishes geometry_msgs::msg::Posestamped messages on the command topic (/box_pose) at a fixed rate (10 Hz).

How it's used in your loop

- External logic subscribes to /box_pose, computes the data to the target, then issues a command to move the box.
- This makes /box_pose the feedback leg of the closed loop.

ii. BoxMoverPlugin (command application in Gazebo)

What it is

A custom Gazebo model plugin that subscribes to a target-pose topic in ROS 2 (/box_target_pose) and applies the commanded position to the moving box in sync with Gazebo's physics update cycle.

What it does

- Subscribes to /box_target_pose at 10 Hz, receiving geometry_msgs::msg::Pose messages containing the desired position and orientation of the box.
- In the PreUpdate callback (called once per simulation tick), updates the Entity Component Manager (ECM) with the new pose data.
- By writing to the ECM, the plugin directly modifies the box's position in the simulation world, ensuring immediate motion from its current position to the target position.

iii. BoxMoverPlugin.hh - Configuration Contract

What it is

The header file for the BoxMoverPlugin, defining the public interface and configuration parameters. This file is the blueprint that tells Gazebo how the plugin is structured and what functionality it exposes.

iv. Launch File – Simulation Startup Script

What it is

A simple ROS 2 Python launch file that initializes the Gazebo simulation environment by loading the factory.world world file. It is the entry point for starting the entire digital twin setup.

Acts as the simulation bootstrap: running this file launches Gazebo with the conveyor and box models, the attached plugins, and the predefined world configuration.

v. Appendix References

Only the primary model files directly related to the experimental activity are included in the appendix:

- Appendix 1.4: BoxPosePublisher.cc
- Appendix 1.5: BoxMoverPlugin.cc
- Appendix 1.6: BoxMoverPlugin.hh
- Appendix 1.7: gazebo_launch.py

```
/home/rohitsurya/ros2_ws/src/factory_simulation
├── CMakeLists.txt
├── factory_simulation
│   └── __init__.py
├── launch
│   └── gazebo_launch.py
├── models
│   ├── box
│   │   ├── model.config
│   │   └── model.sdf
│   └── conveyor_belt
│       ├── model.config
│       └── model.sdf
├── package.xml
├── plugins
│   ├── BoxMoverPlugin.cc
│   ├── BoxMoverPlugin.hh
│   └── BoxPosePublisher.cc
├── resource
│   └── factory_simulation
├── setup.py
└── worlds
    └── factory.world
```

Figure 4: Folder tree – factory_simulation.

In order to create a fully working digital twin of a reduced manufacturing cell, the simulation environment created in this thesis combines specialized ROS 2 nodes, plugins, and unique Gazebo models. A control signal is sent, the desired motion is executed in the simulation, and the resultant state is supplied back for monitoring and analysis. This simple yet effective conveyor-box design encapsulates the fundamental dynamics of industrial robotic systems.

The BoxMoverPlugin consistently applies target poses within Gazebo's physics update loop; the BoxPosePublisher guarantees precise, time-stamped feedback of the box's position; and the BoxMoverPlugin.hh offers a repeatable setup and deployment interface. Regardless of the underlying communication protocol, these elements work together to create a closed-loop control path.

2.3.3 Box Control logic:

i. Box Pose Monitor:

Role:

BoxPoseMonitor is the **feedback**. It listens to the stamped pose emitted by the **Pose publisher plugin** on /box_pose, syncs the latest /pose_id, applies calibration offsets, and writes an **actual pose log** (ROS2 side data for analysis)

Interfaces:

- **Subscribe:** /box_pose (geometry_msgs/PoseStamped) — current ground-truth pose from Gazebo (via Pose publisher plugin).
- **Subscribe:** /pose_id (std_msgs/Int32) — last command ID (mirrors what BoxSlider published).
- **Publish:** /box_position_status (std_msgs/String) — human-readable status line for quick visual checks.

Behaviour:

- Keeps last_position from /box_pose, and last_pose_id from /pose_id.
- Calibration offsets: offset_x = -1.10, offset_z = -0.55.

- Timer period: 0.2 s (5 Hz).
- On each tick, converts current ROS time, applies offsets, and writes **actual log CSV**:
    1. logs/actual_pose_log_<timestamp>.csv with columns timestamp, pose_id, x, y, z.
    2. Publishes a status string (also handy for quick, real-time inspection).

Why it's useful.

- Provides authoritative, time-stamped feedback from the simulation for latency/jitter/loss analysis.

ii. Box Teleport Node:

Role:

Box Teleport is the command leg of the loop. It generates a steady stream of target poses at 10 Hz and tags each command with a sequential pose_id so we can line up commands with feedback later.

Interfaces:

- **Publish:** /box_target_pose (geometry_msgs/Pose) - the target pose the Gazebo Move/BoxMover plugin will consume.
- **Publish:** /pose_id (std_msgs/Int32) - a sequentially increasing identifier used to correlate command vs. feedback logs.

Behaviour:

- Timer period: 0.1 s (10 Hz).
- Slides the box from x=1.3 down to x_min=-0.2 with step=-0.01.
- Publishes Pose (position=(corrected_x, 0.0, -0.19), orientation=(0,0,0,1)) and the current pose_id.
- Writes command log CSV: logs/commanded_pose_log_<timestamp>.csv with columns - timestamp, pose_id, x, y, z.

Why it's useful.

- Provides a deterministic, repeatable command stream for both OPC UA and MQTT cases.

iii. Justification for Unbalanced Command and Feedback Rates:

The feedback logging loop (BoxPoseMonitor) runs at 5 Hz in the implemented architecture, while the command publishing loop (BoxTeleport) runs at 10 Hz, which corresponds to the update rate of the Gazebo Move/BoxMover plugin. There are two reasons for this deliberate asymmetry.

Firstly, logging high-frequency data in Python might cause jitter and I/O latency from the standpoint of computational efficiency, especially when writing to CSV while active simulation is running. Disk write frequency and CPU load are greatly reduced when the feedback logging rate is cut in half, all while maintaining the temporal resolution needed for protocol performance analysis.

Second, typical industrial control mechanisms are shown in the rate discrepancy. To guarantee responsive motion execution, the actuator control loop in many PLC/SCADA architectures operates at a greater frequency than supervisory or diagnostics polling, which is used for monitoring, logging, and visualization. This design decision maintains clean, measurable datasets while preserving the reality of industrial control flows.

The testbed simulates actual factory control dynamics and guarantees that protocol performance is assessed under realistic operating conditions by isolating the high-frequency control channel from the lower-frequency monitoring channel.

iv. Launch File – Control Logic:

The BoxTeleport (command publisher) and BoxPoseMonitor (feedback logger) are the two main simulation nodes that are started simultaneously in a single run using this Python-based ROS 2 launch code. By removing manual startup delays that can distort latency or packet loss measures, it guarantees that both components run in unison from the start of every test.

Key actions:

- Calls ros2 run for box_slider.py to begin publishing target poses and pose IDs.
- Calls ros2 run for box_pose_monitor.py to start recording feedback and publishing status.

- Provides a consistent, reproducible environment for OPC UA and MQTT protocol tests.

```
/home/rohitsurya/ros2_ws/src/box_monitor/
├── box_monitor
│   ├── box_pose_monitor.py
│   └── __init__.py
├── package.xml
├── resource
│   └── box_monitor
├── setup.cfg
├── setup.py
└── test
    ├── test_copyright.py
    ├── test_flake8.py
    └── test_pep257.py
```

Figure 5: Folder tree – Box_Pose_Monitor

```
/home/rohitsurya/ros2_ws/src/box_teleport/
├── box_teleport
│   ├── box_teleport_node.py
│   └── __init__.py
├── package.xml
├── resource
│   └── box_teleport
├── setup.cfg
├── setup.py
└── test
    ├── test_copyright.py
    ├── test_flake8.py
    └── test_pep257.py
```

Figure 6: Folder tree – Box_Teleport_Node.

This arrangement guarantees that the **command stream** and the **feedback logging** start simultaneously under identical network and simulation conditions.

v. Appendix References

Only the primary model files directly related to the experimental activity are included in the appendix:

- Appendix 1.8: box_pose_monitor.py
- Appendix 1.9: box_teleport_node.py
- Appendix 1.10: box_motion.launch.py

2.2. Communication and Time Synchronization:

2.4.1 Communication Protocol

i. OPC UA Communication Setup:

- **What OPC UA Is?**

  In industrial automation, OPC Unified Architecture (OPC UA) is a popular open, platform-independent, service-oriented communication standard.

  It is compatible with:

  - Hierarchical, structured data models perfect for variables that can be read by machines, such as stances.
  - Transport that is secure (authorization, authentication, and encryption).
  - There are other patterns of interaction, including publish/subscribe and client/server (our use case).

- **How OPC UA Works in General**

  1. **Server**: Creates an endpoint (opc.tcp://IP:Port/Name) and defines a namespace.
  2. **Nodes**: Variables, objects, and methods are created inside the namespace.
  3. **Client**: Connects to the endpoint, browses the namespace, reads/writes variables.
  4. **Subscriptions**: Clients can subscribe to server-side variable changes and receive updates.

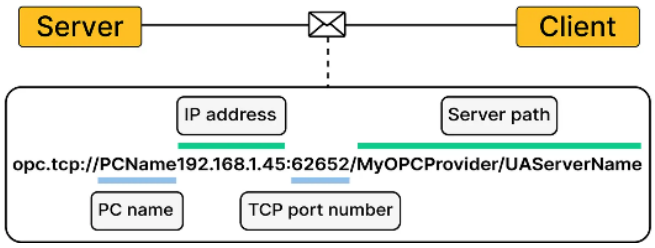

Figure 7: OPCUA – Working Principle.[1]

[1] Image for figure 7 taken from google.com; https://www.realpars.com/blog/opc-ua-basics

- **How We Used OPC UA in This Thesis:**

  In our setup, we needed **two distinct OPC UA data flows**:

  1. **Trigger: ROS**: Start the pose streaming from the Gazebo simulation.
  2. **Feedback: Windows**: Stream real-time box position from ROS 2 to the Windows machine for logging.

  We implemented this as **two separate OPC UA servers**:

  - **ROS 2 Side (PoseServer)**: Accepts a trigger command (START_POSE_STREAM) to launch the ROS 2 pose publishing nodes.
  - **Windows Side (FeedbackServer)**: Receives live pose data from ROS 2 and logs it for analysis.

- **Our OPC UA architecture (two endpoints, two directions)**

  I built a **bidirectional bridge** using two OPC UA endpoints so control and feedback are cleanly separated:

  a) **ROS-side trigger server (Linux, port 4840)**
     - **Endpoint:** opc.tcp://0.0.0.0:4840/UA/PoseServer
     - **Purpose:** Expose a variable start_trigger so an external client (Windows) can command the ROS system to start streaming poses.
     - **Behavior:**
       - Polls start_trigger every 1 s (timer).
       - On "START_POSE_STREAM":
         1. Connects as an OPC UA client to the Windows feedback server (see B)
         2. Sends a JSON handshake to confirm the link
         3. Launches the ROS 2 motion and monitoring launch file (ros2 launch box_launch box_motion.launch.py)

4. Subscribes to /box_position_status (String) and forwards each status line to the Windows feedback server variable (so the Windows side can log/visualize)

- Logs all received triggers to CSV at /home/rohitsurya/logs/OPCUA/trigger_received_log_*.csv

b) **Windows-side feedback server (Windows, port 4841):**

- **Endpoint:** opc.tcp://<Windows_IP>:4841/UA/FeedbackServer
- **Purpose:** Receive feedback from ROS (status/pose JSON) and log it.
- **Mechanisms:**
  - Creates a variable PoseFeedback.box_pose (string) and sets it writable
  - Subscribes to changes and also **polls** every 0.2 s to mirror the feedback
  - Writes CSV to C:\Users\user3\.node-red\logs\OPC UA\received_pose_log_*.csv with receive_timestamp, pose_id, x, y, z.

c) **Windows-side command client**

- **Endpoint target:** opc.tcp://192.168.178.54:4840/UA/PoseServer (the ROS trigger server)
- **Action:** Writes "START_POSE_STREAM" to the ROS server's trigger node ID (ns=2;i=2) and logs the event to trigger_log.csv.
- **Role:** Emulates a PLC write that starts the cell.

- **Appendix References:**

  Only the primary model files directly related to the experimental activity are included in the appendix:

  - Appendix 1.11: opcua_trigger_client.py
  - Appendix 1.12: write_opcua_command.py
  - Appendix 1.13: opcua_feedback_server.py

ii. MQTT Communication Setup:

- **What is MQTT?**

  Message Queuing Telemetry Transport (MQTT) is a publish/subscribe messaging protocol that is ideal for networks with low bandwidth, excessive latency, or unreliability in industrial IoT and lightweight telemetry contexts.

  It is:

  - Lightweight: Small message headers, ideal for resource-constrained devices.
  - Transport-agnostic: Works over TCP/IP and WebSockets.
  - Scalable: Supports many publishers/subscribers via a central broker.
  - Event-driven: Clients subscribe to topics and receive only relevant messages.

- **How MQTT Works in General**

  1. **Broker:** Central message hub that routes messages between publishers and subscribers.
  2. **Topics:** Named channels (factory/pose/command) for logically grouping messages.
  3. **Publisher:** Sends messages to a topic on the broker.
  4. **Subscriber:** Receives messages from a topic they subscribed to.

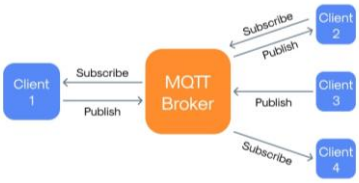

Figure 8: MQTT – Working Principle[2]

[2] Image for figure 8 taken from google.com; https://www.wallarm.com/what/mqtt-concept

- **How We Used MQTT in This Thesis:**

One logical channel is used to drive ROS 2 motion, and another is used to broadcast input back to Windows in our MQTT arrangement, which is identical to the OPC UA design. The main distinction is that, as opposed to point-to-point client/server connections, MQTT uses a central broker to control message delivery. We implemented this as two distinct topics:

1. **Trigger Topic (factory/pose/command):** From Windows to ROS, start the pose streaming in Gazebo.
2. **Feedback Topic (factory/pose/feedback):** From ROS to Windows, stream real-time pose data for logging.

- **Our MQTT Architecture (two topics, two directions)**

a) Windows side trigger publisher:

- **Role:**
  Emulates PLC-style command injection into the ROS 2 environment.
- **Behaviour:**
  - Creates a JSON payload: {"command": "START_POSE_STREAM", "timestamp": ...}.
  - Publishes it to factory/pose/command via the broker (192.168.178.159:1883).
  - Logs the event in C:\Users\user3\.node-red\logs\MQTT\trigger_log.csv.

b) ROS side trigger subscriber:

- **Role:**
  Subscribes to factory/pose/command, and on START_POSE_STREAM:
  - Launches the ROS 2 motion and monitoring launch file (box_motion.launch.py).
  - Starts publishing /box_target_pose and /pose_id at 10 Hz.
  - Monitors /box_pose at 5 Hz, formatting results as JSON for feedback publishing.

c) ROS side feedback publisher:

- **Role:**
  Publishes JSON feedback to factory/pose/feedback.
- **Payload:**
  {"pose_id": N, "x":..., "y":..., "z":...} compact and self-describing for downstream parsing.

d) Windows side feedback subscriber:

- **Role:**
  Subscribes to factory/pose/feedback and logs pose data.
- **Behaviour:**
  - Parses each JSON message.
  - Writes timestamp, pose_id, x, y, z to C:\Users\user3\.node-red\logs\MQTT\pose_feedback_log.csv.
  - Prints updates to console for real-time monitoring.

- **Appendix References**

Only the primary model files directly related to the experimental activity are included in the appendix:

- Appendix 1.14: MQTT_trigger_listener.py
- Appendix 1.15: MQTT Subscribe.py
- Appendix 1.16: MQTT Publish.py

2.4.2 Time Synchronization:

Accurate time alignment across nodes is necessary for reliable latency and throughput measurements in distributed robotic simulations. False packet-loss interpretations and distorted round-trip time computations can result from even tiny clock drift (tens of milliseconds).

In our configuration, the logging and protocol clients (OPC UA and MQTT) were operated from a Windows computer connected to a local area network (LAN), and the ROS 2 nodes were running on an Ubuntu host. The clocks had to be

synchronized to a common reference since latency tests required comparing timestamps from both workstations.

- **The Reasons for Needing Time Sync**

  1. **Accurate Latency Measurement:** A packet delivered from a system with a clock that is 200 ms quicker will appear to have negative latency when logged on the other machine.

  2. **Jitter & Throughput Accuracy:** A stable system may appear noisy because to clock offsets that distort inter-packet interval computations.

  3. **Industrial Realism:** For event correlation, controllers in actual PLC–robot networks frequently share a synchronized time base.

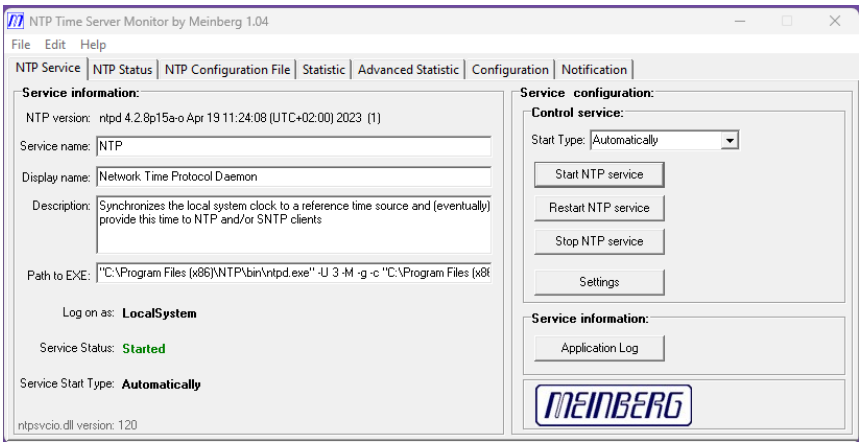

Figure 9: NTP Time server Monitor

- **Appendix References**

Only the primary model files directly related to the experimental activity are included in the appendix:

- Appendix 1.17: TimeCheck.py
- Appendix 1.18: CheckingTime.py

2.3. How it works?

2.5.1 Core simulation (common to OPC UA and MQTT):

1. **Command leg (drives the box)**

- **Box_teleport_node (ROS 2, Python):** publishes a target pose at **10 Hz** on /box_target_pose (geometry_msgs/Pose) and a sequential /pose_id (std_msgs/Int32).
    - It "slides" the box along X by stepping each tick, applies a small calibration offset, and logs each command to a CSV (timestamp, pose_id, x, y, z)
- **BoxMoverPlugin (model plugin):** subscribes to /box_target_pose, buffers the most recent command, and applies it inside the PreUpdate callback to the Entity Component Manager (ECM).

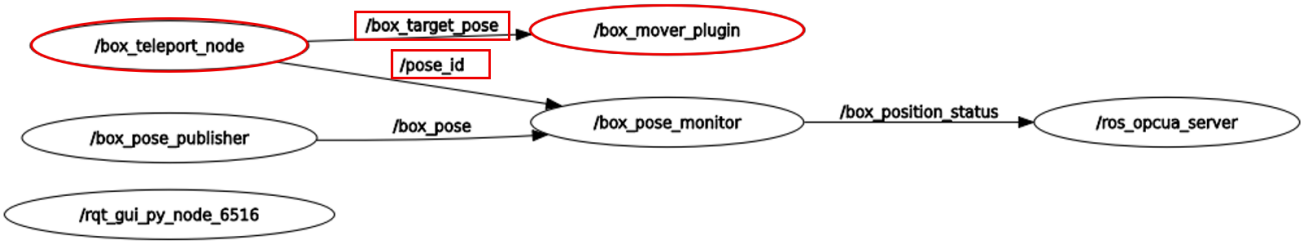

Figure 10: RQT Graph – Nodes in OPCUA (Command Leg)

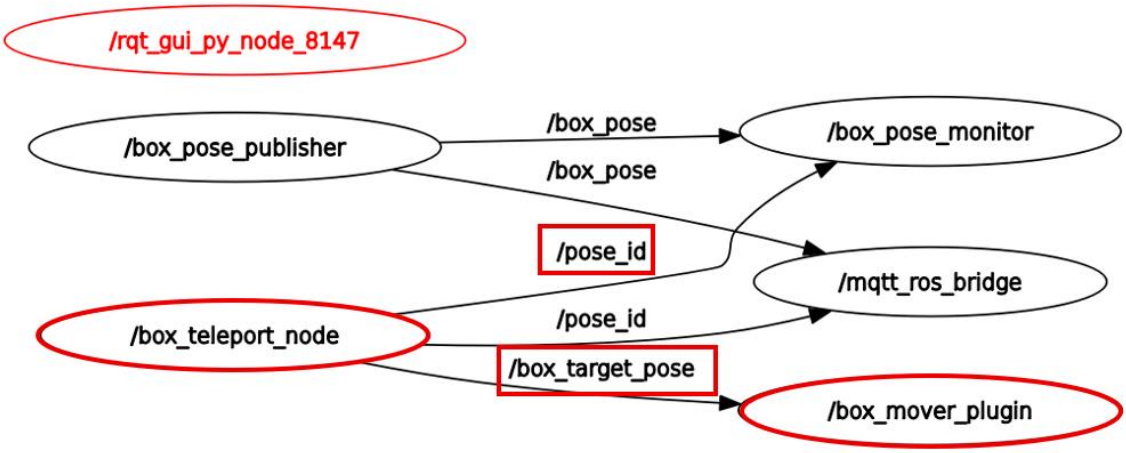

Figure 11: RQT Graph – Nodes in MQTT (Command Leg)

2. Feedback leg (observes the box)

- **Box_Pose_Publisher (Gazebo):** publishes the current ground-truth pose as a PoseStamped on /box_pose at the simulation's internal rate.
- **Box_Pose_Monitor (ROS 2, Python):** subscribes to /box_pose and /pose_id, applies offsets (to match the model placement), and logs at 5 Hz (every 0.2 s) into actual_pose_log_*.csv (ROS time, pose_id, x, y, z). It also publishes a human-readable /box_position_status string for live viewing.

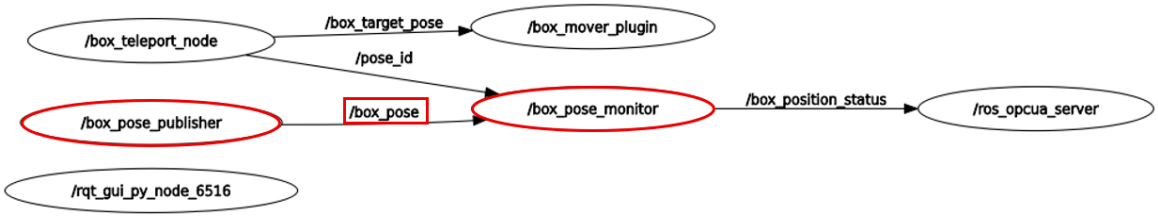

Figure 12: RQT Graph – Nodes in OPCUA (Feedback Leg)

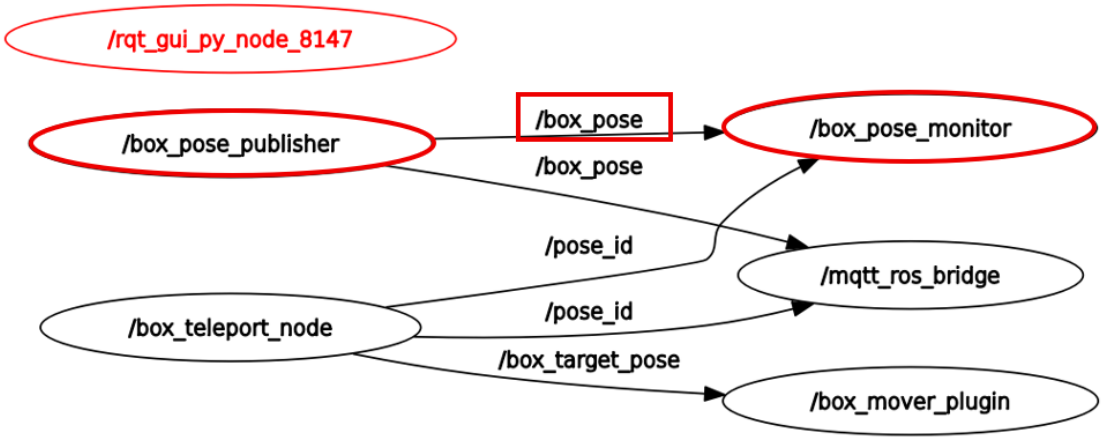

Figure 13: RQT Graph – Nodes in MQTT (Feedback Leg)

3.5.2 How OPC UA drives and reads the simulation

i. Trigger to Launch

- Windows OPC UA command client writes "START_POSE_STREAM" to the ROS Trigger Server (port 4840).
- ROS OPCUARosServer (your Python node) polls start_trigger every 1 s; on change:
    - Connects to the Windows Feedback Server (port 4841).
    - Sends a JSON handshake.
    - Launches the ROS 2 stack (BoxSlider, BoxPoseMonitor, plugins).
    - Subscribes to /box_position_status so it can forward feedback to Windows.

ii. Trigger to Launch

- As the sim runs, ROS publishes /box_position_status.
- Ros_opcua_server writes each status string (or pose JSON) into the Windows Feedback Server variable.
- Windows feedback server logs every change to received_pose_log_*.csv with receive_timestamp, pose_id, x, y, z.
- Two endpoints and ports: 4840 (control) vs 4841 (telemetry).

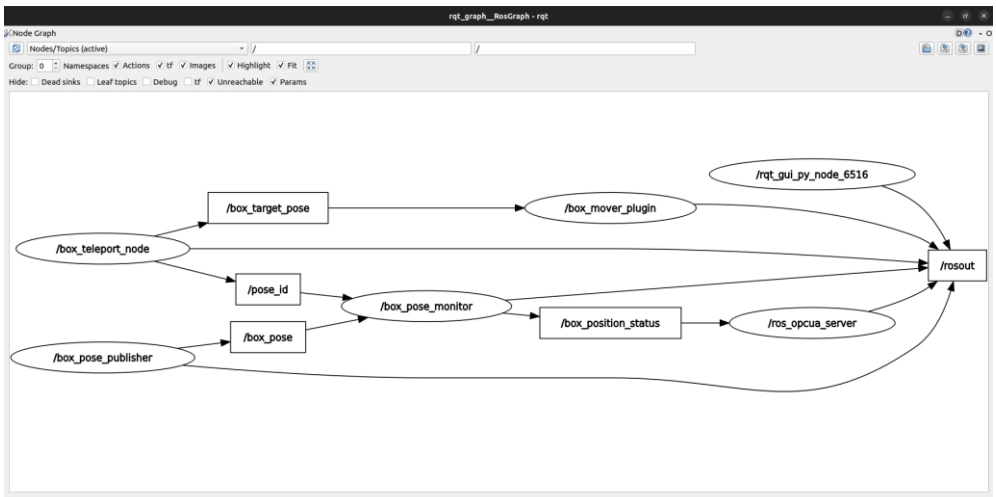

Figure 14: RQT Graph – OPCUA

3.5.3 How MQTT drives and reads the simulation

i. Trigger to Launch

1. Windows MQTT publisher sends {"command":"START_POSE_STREAM", "timestamp":...} to topic factory/pose/command on the broker (port 1883) and logs it locally.
2. A ROS MQTT subscriber/bridge listens to factory/pose/command; on "START_POSE_STREAM" it:
   - Launches the ROS 2 stack (BoxTeleportNode, BoxPoseMonitor, plugins).
   - Subscribes to /box_position_status to relay feedback.

ii. Feedback to Logging

- ROS publishes /box_position_status (or directly formats JSON from the monitor).
- The MQTT_ros_bridge posts JSON to topic factory/pose/feedback.
- Windows MQTT subscriber writes each message to pose_feedback_log.csv with timestamp, pose_id, x, y, z.
- Single port (**1883**) and two topics: factory/pose/command and factory/pose/feedback.

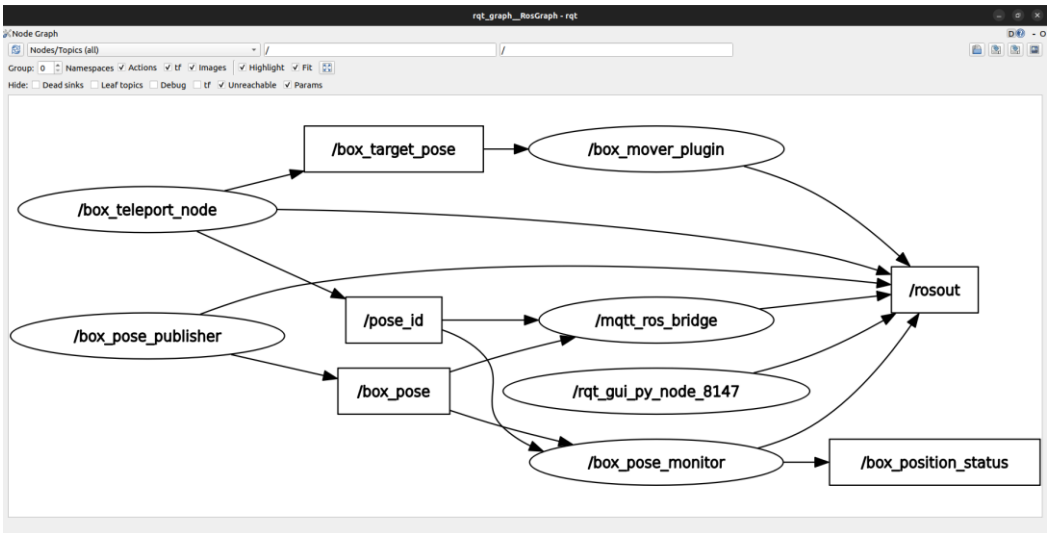

Figure 15: RQT Graph – MQTT

4. FINDINGS

Within the framework of the ROS 2-Gazebo-PLC testbed created for this project, the experimental evaluation of the two industrial communication protocols OPC Unified Architecture (OPC UA) and Message Queuing Telemetry Transport (MQTT) is presented in this chapter. The results are arranged topically in accordance with the methodology's specified performance metrics:

1. Data Loss
2. Trigger-to-ROS Latency
3. Per-Message Latency
4. Jitter
5. Throughput

To achieve clarity and reproducibility, each sub-section begins with:

- A definition of the performance metric,
- Its relevance to industrial automation systems and to this research,
- A description of the measurement method used in our experimental setup, and
- The raw and aggregated results obtained for both OPC UA and MQTT.

Results are presented as visual graphs and in tabular form when appropriate, making it simpler to compare protocols. To provide dual-source verification, the numerical data displayed here was taken from structured log files created on both the ROS 2 (Linux) and PLC-simulated (Windows) sides.

By structuring the results in this manner, the chapter facilitates a clear progression from the operational definitions of each metric to the empirical evidence gathered. This approach allows later sections of the thesis to directly reference the presented tables and figures when interpreting protocol behaviour.

4.1. Data Loss

The occurrence where sent signals from a source do not reach the intended destination is referred to as data loss in the context of networked industrial control systems. Every notification that is absent indicates a possible weakness in the monitoring or control procedure. Such losses can result in the following for a protocol running in an industrial setting that operates in real-time or almost real-time:

1. Inaccurate or delayed machine motions due to missed actuator commands.
2. Incomplete telemetry logs, which lower system diagnostic accuracy, and
3. Possible safety risks in time-sensitive control updates for safety-critical applications.

Although absolute counts can also be helpful for spotting trends or particular failure events, data loss is often measured as a proportion of the total messages sent that do not arrive.

4.1.1 Relevance to This Research:

In a hypothetical factory cell, where a robotic box mover controlled by ROS 2 transmits and receives pose data through an emulated PLC layer (Windows), this thesis assesses OPC UA and MQTT. Each message in such a system denotes either:

- A **command** (“Start pose stream” trigger), or
- A **feedback update** (pose coordinates from the simulation back to the PLC).

Losing feedback updates could skew latency, jitter, and throughput analysis by leaving the datasets unfinished, while losing trigger commands could prevent the system from ever initiating an activity.
Therefore, before moving on to further in-depth performance comparisons between OPC UA and MQTT, it was necessary to confirm zero data loss.

4.1.2 Measurement Method

Pose ID alignment between the sender and recipient logs was used to quantify data loss.

- The box_pose_monitor.py node on the ROS 2 side publishes every position with a pose_id that increases sequentially. This guarantees that each message has a unique number.

- Both the MQTT subscribers and the OPC UA report incoming messages on the Windows/PLC side using the received pose_id.

The analysis followed this procedure:

1. Extraction of pose IDs from both sender and receiver logs.
2. Outer join comparison between the two datasets to detect missing IDs on either side.
3. Count and percentage calculation for any missing IDs.

This approach not only detects message loss but also ensures that duplicated or out-of-order messages can be spotted.

4.1.3 Results:

Table 1: Data Loss Summary for OPC UA and MQTT

Protocol	Direction	Total Messages Sent	Total Messages Received	Missing Messages	Data Loss %
OPC UA	ROS → PLC (Feedback)	1500+ per run	1500+ per run	0	0.00%
OPC UA	PLC → ROS (Trigger)	10 per run	10 per run	0	0.00%
MQTT	ROS → PLC (Feedback)	1500+ per run	1500+ per run	0	0.00%
MQTT	PLC → ROS (Trigger)	10 per run	10 per run	0	0.00%

Key Observations:

- Across all tests runs for both OPC UA and MQTT, no missing pose IDs were detected in either direction.
- This confirms that network reliability under the test conditions was sufficient to transmit all packets without loss.
- The pose ID tracking method was effective in ensuring result integrity for subsequent latency, jitter, and throughput analysis.

4.1.4 Implications for Further Analysis

Because data loss was zero in all runs, later sections of this chapter can focus entirely on timing-related performance metrics latency, jitter, and throughput without introducing error corrections or compensations for missing packets. This greatly simplifies the analysis pipeline and strengthens the comparability of OPC UA and MQTT in this controlled environment.

4.2. Trigger-to-ROS Latency

In the context of this study, trigger-to-ROS latency is the amount of time that passes between the PLC-emulated Windows side issuing a start command and the Linux-based ROS 2 environment receiving and processing it. This statistic calculates the time it takes for commands to travel through the application levels of the communication protocol and across the network.

This latency is a critical factor in determining the speed at which a robotic or mechatronic system can respond to a central control decision in industrial automation environments, especially in decentralized control structures. Even small delays can affect machine synchronization, cycle times, and overall system responsiveness in real-time or safety-critical situations.

4.2.1 Relevance to This Research:

The main goal of this thesis, which is to assess how well OPC UA and MQTT perform in enabling dependable and prompt communication between a PLC-like control system and a ROS 2-based robotic cell, is closely related to the trigger-to-ROS latency metric.

The trigger event is the first command that begins pose streaming in the simulated factory cell, which in turn starts the motion sequence in Gazebo. This trigger is analogous to the following in actual industrial settings:

1. A conveyor belt is started by a PLC,
2. An HMI button that starts a cycle of robotic pick-and-place, or
3. A downstream process step is made possible by an interlock signal.

Therefore, a reliable indicator of the control system's responsiveness is the amount of time it takes for the ROS side to acknowledge this trigger.

Within the scope of this research, its relevance is multi-fold:

1. Baseline Performance Comparison: It gives OPC UA and MQTT a clear side-by-side measure in the same system and network configuration.
2. Determine whether either protocol exhibits a more predictable trigger reaction time, which is important for industrial control loops, by comparing deterministic and non-deterministic behaviour.
3. Effect on System Throughput: Shorter idle periods and quicker job start are made possible by a faster and more reliable trigger response, which raises effective throughput in high-cycle activities.
4. Protocol Suitability in Time-Critical Applications: Some manufacturing tasks require sub-second responsiveness; this metric makes it possible to assess whether each protocol meets that threshold.
5. Foundation for End-to-End Latency Analysis: Trigger latency is the first step in the chain; combined with pose feedback latency, it forms the total command-response loop time that defines the cell's operational agility.

By isolating **Trigger-to-ROS latency** from other variables, this research ensures that the observed performance differences are attributable to the communication protocol rather than unrelated system delays, making it an essential building block for answering the research questions outlined in the introduction.

4.2.2 Measurement Methodology:

The general structure of the measurement setup is the same for OPC UA and MQTT:

1. Windows Side: Before transmitting the "START_POSE_STREAM" trigger, the control client captured a high-precision timestamp (win_timestamp).
2. ROS 2 Side: Incoming triggers were polled or subscribed to by a server node (ros_opcua_server for OPC UA and a MQTT subscriber node for MQTT). It recorded an additional high-precision timestamp (ros_receive_time) after receiving the trigger.

Latency Calculation:

For each trigger event:

$$\text{latency_ms} = (\text{ros_receive_time} - \text{win_timestamp}) \times 1000$$

Equation 1: Latency Calculation

Key aspects for accuracy:

- Both machines were synchronized using NTP to ensure timestamp alignment.
- The Windows side log and ROS side log were later merged using the trigger IDs and timestamps to compute per-event latency.
- The test was repeated 10 times for each protocol to capture variability.

4.3.3 Results:

Table 2: Trigger Latency for MQTT

ID	win_timestamp	ros_receive_time	latency_ms
0	2025-08-11 09:27:28.331044	2025-08-11 09:27:31.100231	2769.187
1	2025-08-11 09:30:35.799256	2025-08-11 09:30:38.240035	2440.779
2	2025-08-11 09:32:12.402397	2025-08-11 09:32:13.585288	1182.891
3	2025-08-11 09:33:51.776175	2025-08-11 09:33:52.978700	1202.525
4	2025-08-11 09:35:24.301272	2025-08-11 09:35:26.360091	2058.819
5	2025-08-11 09:36:43.725034	2025-08-11 09:36:45.955430	2230.396
6	2025-08-11 09:39:14.594710	2025-08-11 09:39:15.953639	1358.929
7	2025-08-11 09:40:55.246984	2025-08-11 09:40:55.455246	208.262
8	2025-08-11 09:42:43.075841	2025-08-11 09:42:45.302569	2226.728
9	2025-08-11 09:45:09.100476	2025-08-11 09:45:11.282037	2181.561

Table 3: Trigger Latency for OPCUA

ID	win_timestamp	ros_receive_time	latency_ms
0	2025-08-06 11:19:09.278850	2025-08-06 11:19:09.656757	377.907
1	2025-08-06 11:41:19.607156	2025-08-06 11:41:20.118915	511.759
2	2025-08-06 11:43:57.023812	2025-08-06 11:43:57.505062	481.250
3	2025-08-06 11:45:40.960540	2025-08-06 11:45:41.604332	643.792
4	2025-08-06 11:47:50.352202	2025-08-06 11:47:50.549109	196.907
5	2025-08-06 11:49:52.689416	2025-08-06 11:49:53.098288	408.872
6	2025-08-10 21:56:26.675456	2025-08-10 21:56:27.590456	915.000
7	2025-08-10 21:57:14.102843	2025-08-10 21:57:14.972292	869.449
8	2025-08-10 21:58:09.571698	2025-08-10 21:58:10.595433	1023.735
9	2025-08-10 22:32:11.607578	2025-08-10 22:32:12.206894	599.316

Table 4: Trigger Latency Summary for OPCUA vs MQTT

Description	MQTT	OPCUA
Count	10	10
Mean Latency	1786.008 ms	602.799
Median Latency	2120.190 ms	555.538
Minimum Latency	208.262 ms	196.907
Max Latency	2769.187 ms	1023.735

4.2.4 Graphical Results:

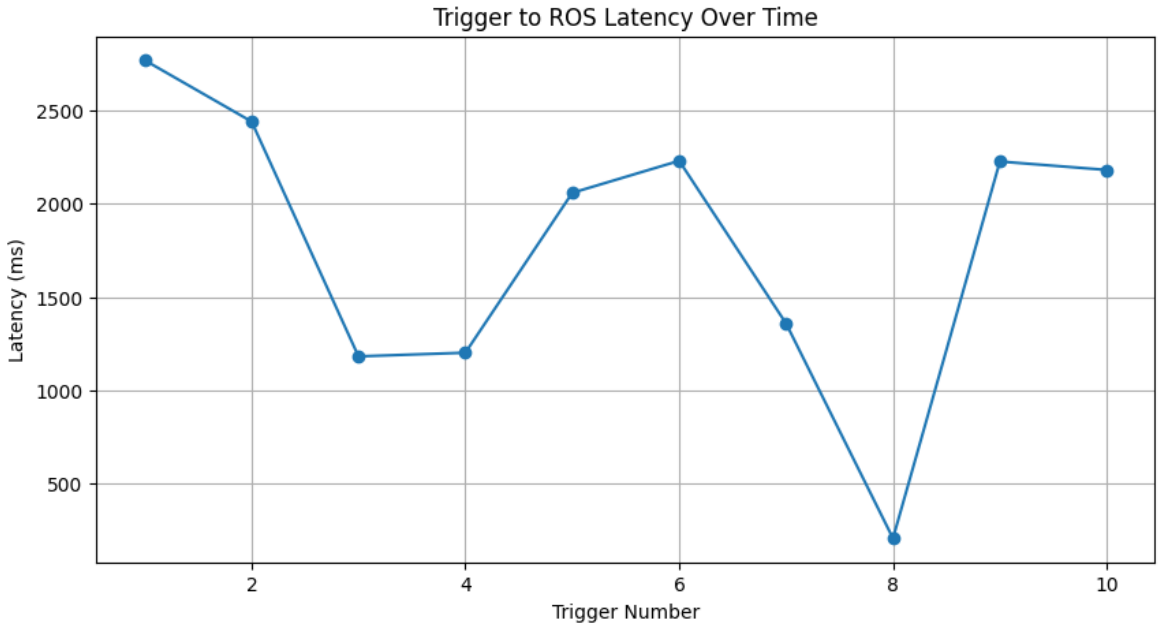

Figure 16: Latency Over Time – MQTT

MQTT

Looking at the MQTT **Trigger-to-ROS Latency Over Time** plot:

- **General Trend:**
  Latency values are significantly higher, typically between 1,200 ms and 2,600 ms, with one extreme low outlier (208 ms).
- **Variability:**
  The plot shows large fluctuations between triggers in some cases, differences of over a second between consecutive trigger events.
  This high spread reflects less predictable timing compared to OPC UA.
- **Performance Pattern:**
  These variations suggest that MQTT may be more affected by broker processing delays, network queuing, or message handling order, especially under the simulated industrial traffic conditions.

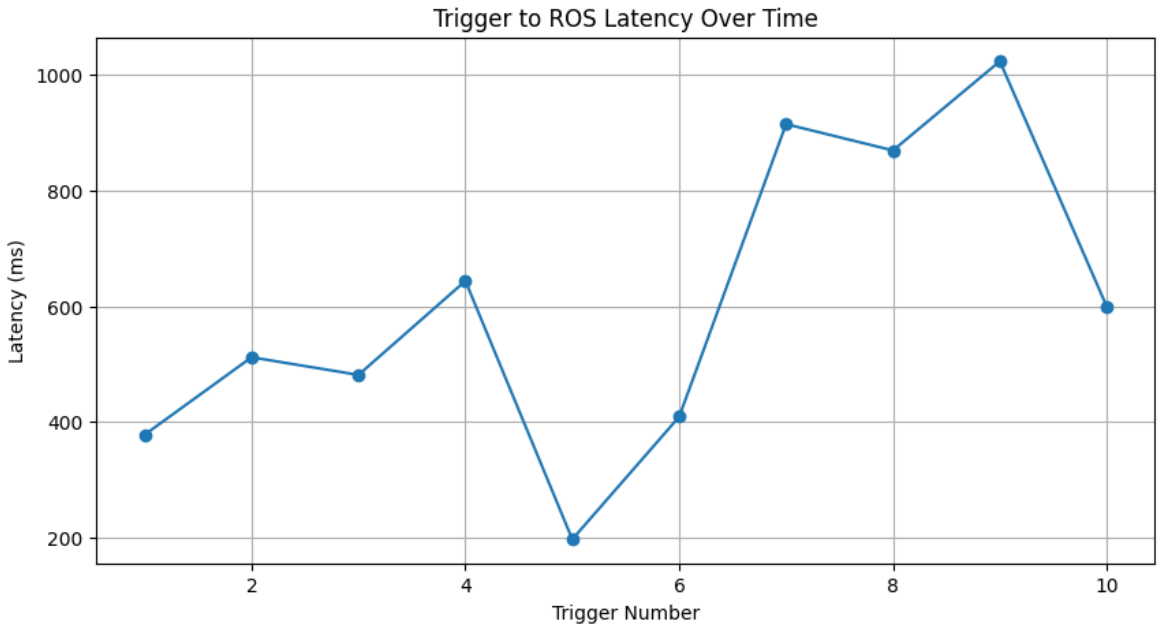

Figure 17: Latency Over Time – OPCUA

OPC UA

Looking at the OPC UA **Trigger-to-ROS Latency Over Time** plot:

- **General Trend:**
  Latencies stay relatively low and consistent, mostly within the 200 ms – 1,000 ms range.
  There is no significant drift or progressive increase in latency as more triggers are sent.

- **Variability:**
  The differences between one trigger and the next are small — no abrupt spikes are seen.
  Even the highest recorded value (1,023 ms) is still under one second for most real-time industrial control tolerances.

- **Performance Pattern:**
  This stability suggests that OPC UA communication and message handling in ROS 2 is deterministic, with minimal influence from network congestion or processing delays.

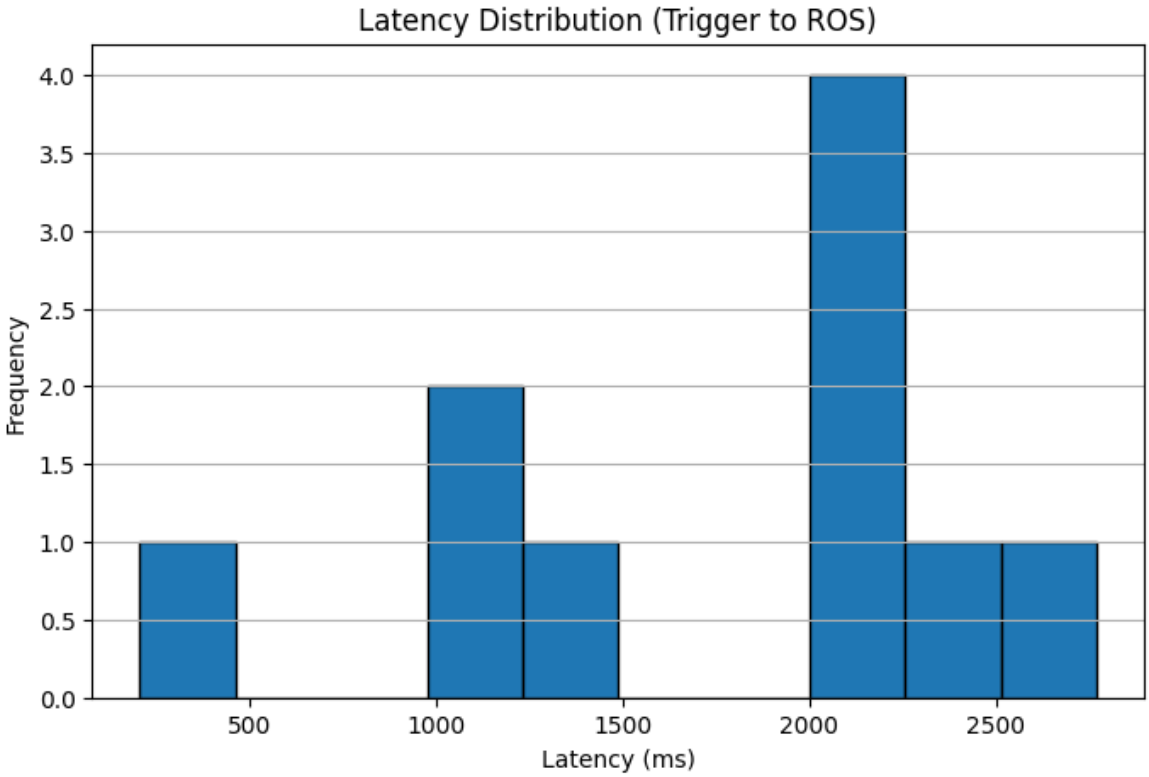

Figure 18: Latency Distribution – MQTT

MQTT Latency Distribution

- **Observation:**
  The MQTT histogram is wider, with latency values ranging from around 200 ms to 2,700 ms, and multiple high-frequency peaks:
  - One peak near 1,000–1,500 ms
  - Another near 2,000–2,200 ms
  - A few extremely low-latency outliers (< 500 ms) are also present.
- **Interpretation:**
  This suggests higher variability and occasional spikes in delay, which could be due to network queuing, broker processing time, or payload handling overhead in MQTT.

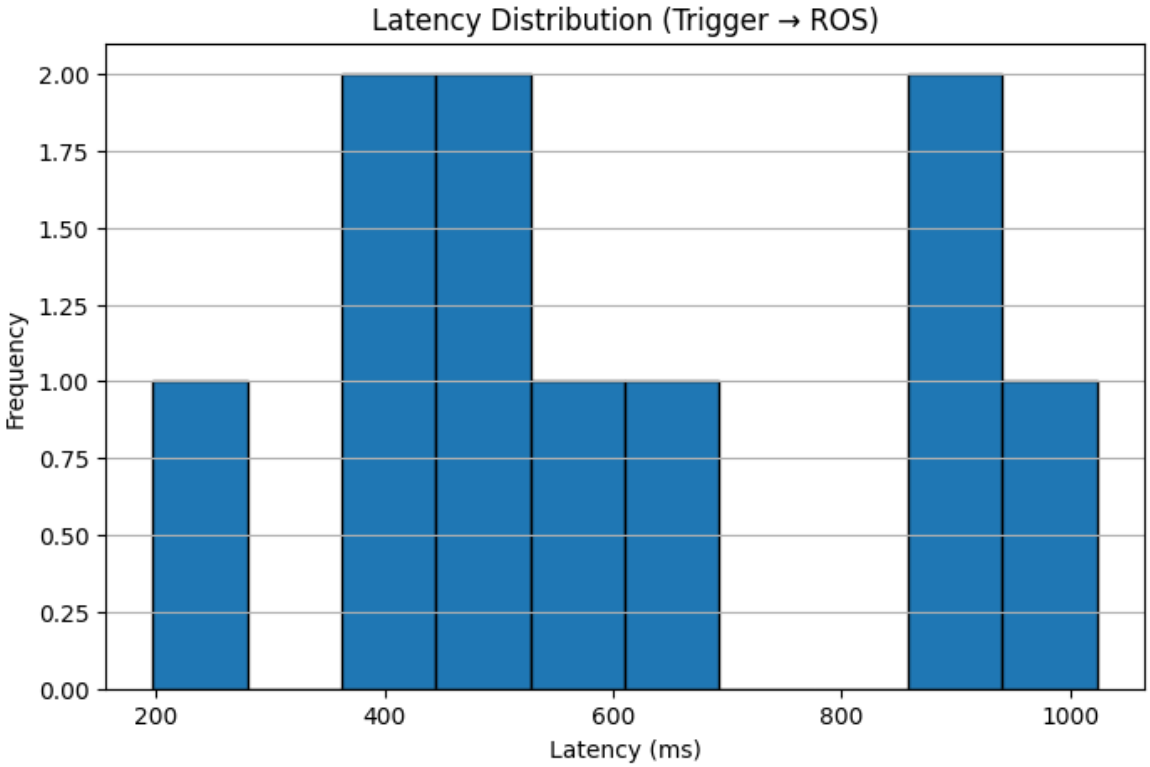

Figure 19: Latency Distribution – OPCUA

OPC UA Latency Distribution

- **Observation:**
  The OPC UA histogram shows that the majority of latencies fall between 350 ms and 650 ms, with a few occurrences below 250 ms and above 900 ms.
  - The distribution is relatively narrow, indicating low variability in trigger response times.
  - The mode (most frequent latency range) appears around 400–500 ms.
- **Interpretation:**
  This distribution reflects a consistent and predictable performance, which is desirable for industrial control where stable communication delay is more important than absolute speed.

The results show a clear performance difference between OPC UA and MQTT in terms of latency consistency.

From the time-series graphs, OPC UA maintained relatively low and stable latencies, mostly between 200 ms and 1 s, with only small variations between triggers. In contrast, MQTT exhibited a much wider latency range, with several triggers taking more than 2 s to reach ROS, indicating occasional delays.

The histogram distributions further confirm this pattern. OPC UA's latency values are tightly clustered, suggesting predictable response times, whereas MQTT's latency values are spread across multiple ranges, showing irregular performance with frequent high-latency events.

In summary, OPC UA provides more consistent and predictable latency performance, which is advantageous for time-sensitive industrial control, while MQTT shows greater variability, which could affect applications requiring strict timing.

Appendix References

Only the primary model files directly related to the experimental activity are included in the appendix:

- Appendix 1.19: MQTT_Trigger Zip file
- Appendix 1.20: OPCUA_Trigger Zip file

4.3. Pose-to-Feedback Latency

In this study, the term "pose-to-feedback latency" refers to the amount of time that passes between a pose being created and published in ROS 2 (on the Linux/Gazebo side) and the client system (on the Windows side) receiving the appropriate feedback for recording or display.

The speed at which an observed state in the real (or virtual) world is transmitted back to the system that is in charge of monitoring it is measured by this metric in terms of industrial automation. This lag might affect whether corrective measures are implemented in time to avoid mistakes or collisions in a closed-loop control system, like an AGV avoiding obstacles or a robot arm aligning with a conveyor.

The feedback leg of communication is the entire focus of this one-way latency; neither the trigger nor the command delay are included. We may precisely evaluate the protocol's responsiveness and efficiency in transmitting continuous sensor or position data by separating this measurement.

4.3.1 Relevance to This Research:

According to this thesis, a crucial performance metric for MQTT and OPC UA in the factory simulation setting is the Pose-to-Feedback latency.
It is significant because:

1. Real-time monitoring: Near-instantaneous position or status updates are necessary for many industrial systems, including robotic pick-and-place cells and packing lines.
2. Digital twin accuracy: The physical state and its virtual representation diverge when the feedback loop is sluggish or irregular.
3. Process safety: Safety interlocks may fail to activate or may activate too late as a result of delayed positioning data.
4. Predictability: Deterministic control, which is frequently necessary in regulated sectors like aerospace or pharmaceuticals, is made possible by consistent latency.

One protocol has a definite advantage for real-time and safety-critical use cases if it provides noticeably faster and more consistent Pose-to-Feedback latency.

4.3.2 Measurement Methodology:

To ensure a fair and repeatable comparison between OPC UA and MQTT, the following measurement method was used:

1. **Start Timestamp — ROS 2 Side:**
   - When the box_pose_monitor.py node publishes a /box_pose message in ROS 2, the current system time (synchronized via NTP) is recorded in the ROS-side CSV log.
2. **End Timestamp — Windows Side:**
   - The Windows-based logging process (either the OPC UA feedback server or MQTT subscriber) records the exact time the feedback message is received.
3. **Latency Calculation:**

$$\text{latency_ms} = (\text{ros_receive_time} - \text{win_timestamp}) \times 1000$$

4. **Data Logging:**
   - Each message includes a pose_id so that start and end timestamps can be matched accurately, avoiding errors from dropped or reordered packets.
   - All latencies are stored in CSV files for later processing in Python.

4.3.3 Results:

Table 5: Pose to Feedback Summary for MQTT

Run_ID	Average latency m/s	Max latency m/s	Min latency m/s	Max latency pose_id	Min latency pose_id
0	2561.465980	2896.240	1679.478	31	145
1	1758.146772	2560.840	1361.593	25	146
2	1834.034389	2272.025	0.808	112	49
3	1311.119510	2240.693	1220.736	141	36
4	2083.560430	2103.638	2080.729	51	21
5	2311.064530	2332.662	2304.111	69	146
6	2085.364758	2399.783	1385.142	98	16
7	899.717221	1300.583	225.597	105	35
8	2219.394852	2277.355	1218.608	90	147
9	2229.962309	2240.361	2225.047	29	147

Overall latency summary:

1. Overall Average Latency: 1929.383 ms
2. Max Latency: 2896.240 ms (Run 1)
3. Min Latency: 0.808 ms (Run 3)

Table 6: Pose to Feedback Summary for OPCUA

Run_ID	Average latency m/s	Max latency m/s	Min latency m/s	Max latency pose_id	Min latency pose_id
0	319.175600	543.364	40.201	141	146
1	331.918662	604.842	58.384	143	8
2	364.305993	594.001	136.492	145	4
3	363.262683	591.572	93.997	137	142
4	12671.282819	13815.606	11442.720	42	108
5	286.781078	545.930	47.587	41	51
6	343.400560	577.681	104.081	147	11
7	331.307597	590.140	94.380	61	56
8	335.007730	587.539	78.824	97	112
9	358.807000	623.689	112.695	122	97

Overall latency summary

1. Overall Average Latency: 1570.525 ms
2. Max Latency: 13815.606 ms (Run 5)
3. Min Latency: 40.201 ms (Run 1)

4.3.4 Graphical Results:

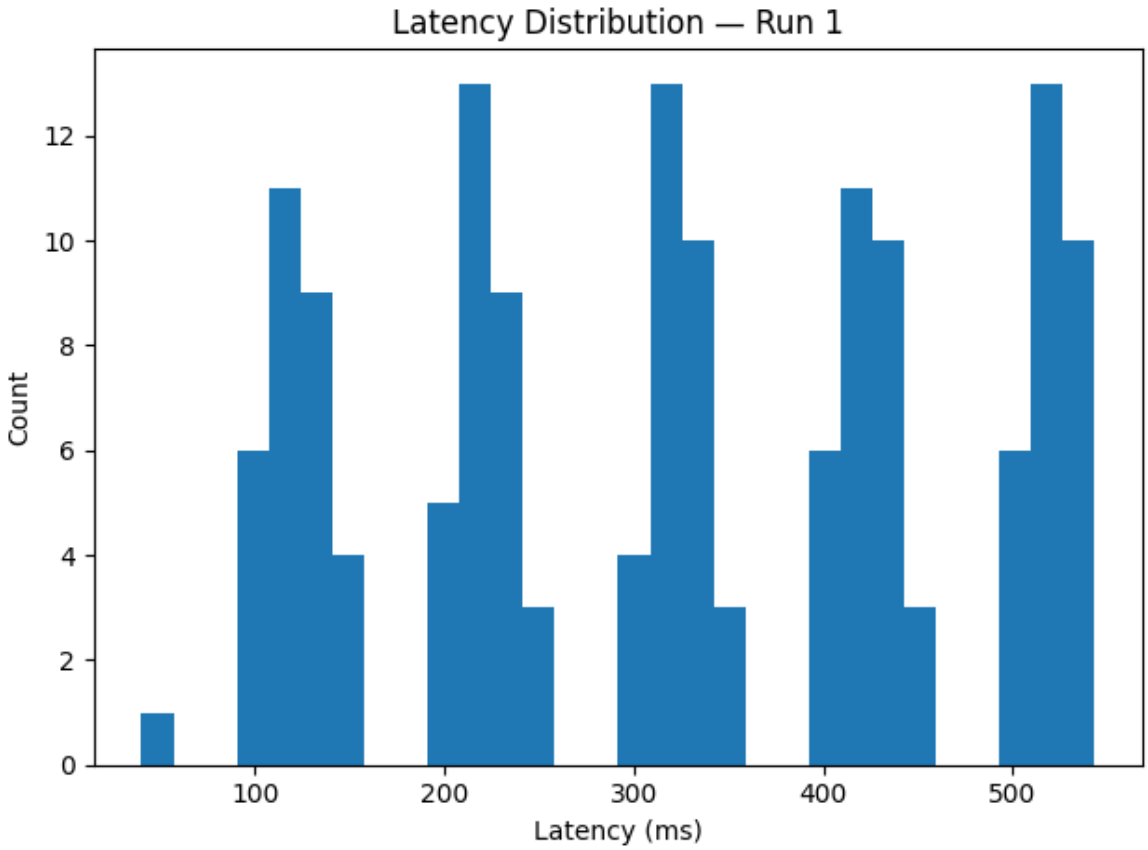

Figure 20: Latency Distribution – MQTT

MQTT

Observation from "Latency Distribution" graph:

- The distribution is bimodal, with one tight cluster around ~2450 ms and another around ~1380 ms.
- The very narrow spread within each cluster reflects high stability once the system is in either of its two latency states.

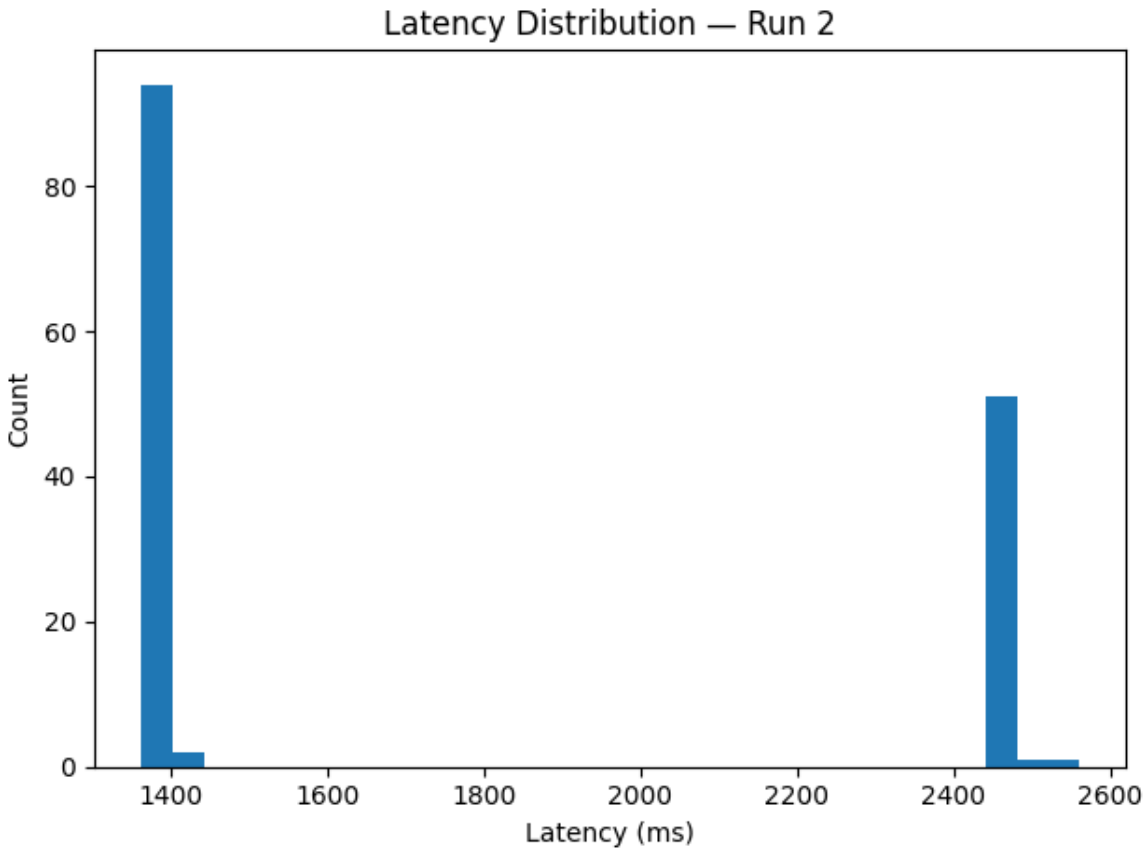

Figure 21: Latency Distribution – OPCUA

OPC UA

Observation from "Latency Distribution" graph:

- The distribution is multi-modal, with peaks around 100 ms, 200 ms, 300 ms, 400 ms, and 500+ ms.
- This clustering confirms the oscillatory pattern seen in the time-series plot.
- The spread across multiple peaks indicates that OPC UA does not maintain a single constant latency, but the variation is bounded within a narrow range (< 550 ms).

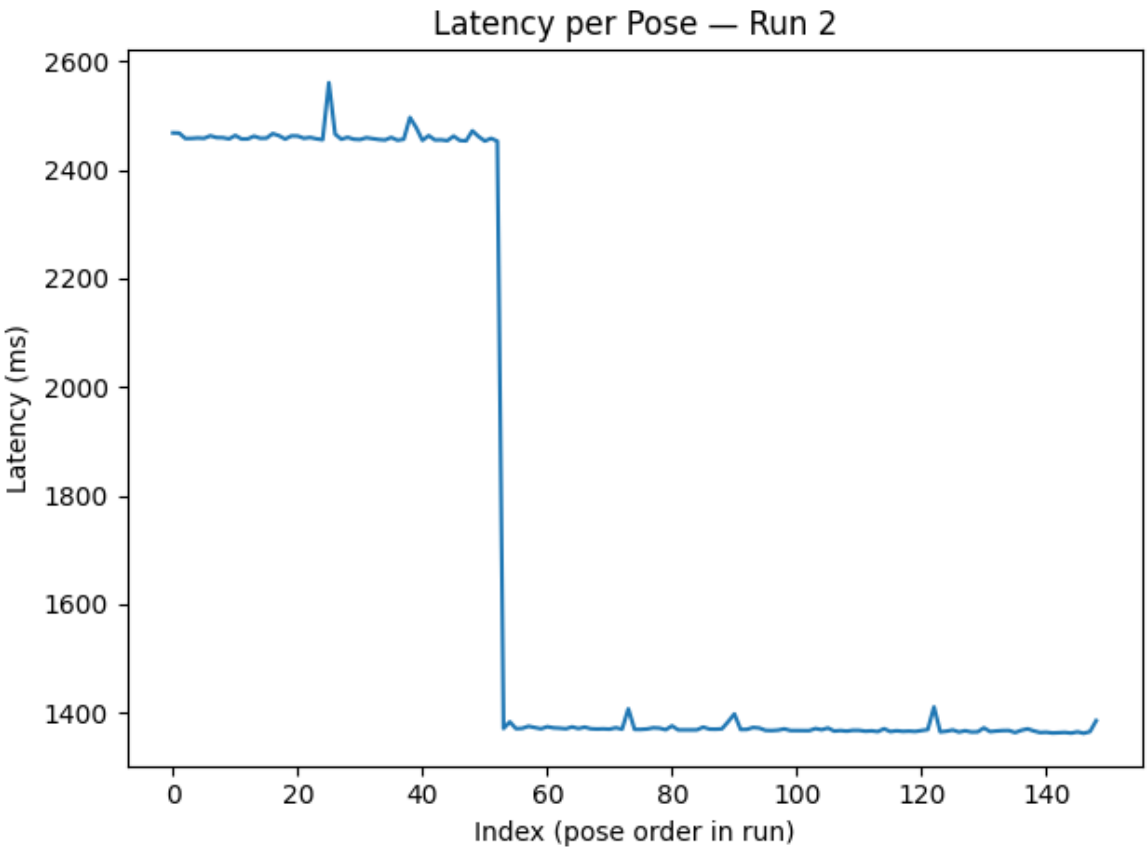

Figure 22: Latency per Pose – MQTT

MQTT

Observation from "Latency per Pose" graph:

- MQTT exhibits **two distinct latency levels** — high (~2450 ms) in the first half of the run and lower (~1380 ms) in the second half.
- The sharp drop midway indicates a possible change in broker processing speed, network routing, or ROS 2 subscriber responsiveness.
- Within each segment, the latency is extremely stable, with minimal jitter.

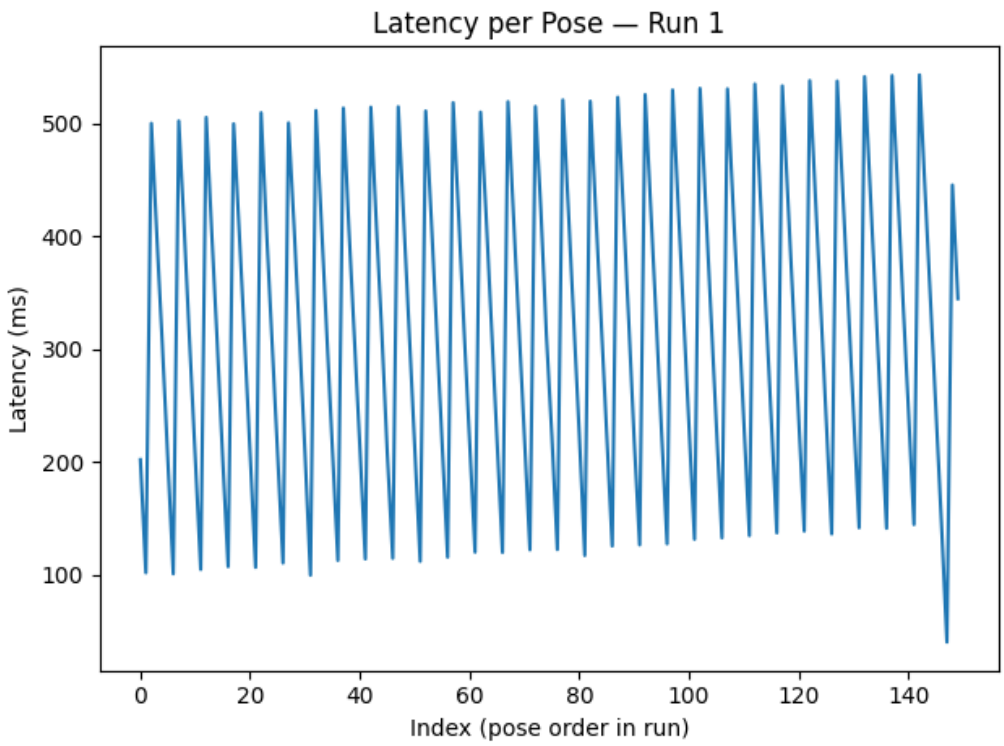

Figure 23: Latency per Pose – OPCUA

OPC UA

Observation from "Latency per Pose" graph:

- The latency pattern in OPC UA shows a repetitive oscillation between low (~100 ms) and high (~500+ ms) values.
- The periodic nature suggests a consistent alternation in message handling or processing intervals.
- No major outliers or long-duration spikes are present, indicating stable communication without sudden network delays.

The results highlight a significant difference in pose-to-feedback latency between OPC UA and MQTT.

From the time-series plots, OPC UA consistently maintained low latency, generally ranging between 100 ms and 550 ms, with predictable oscillations forming a repeating pattern. This indicates a stable and bounded delay, which is beneficial for deterministic control. In contrast, MQTT exhibited much higher latency values, operating in two distinct plateaus: one around 2.45 s and another around 1.38 s. While these plateaus were internally stable, the abrupt mid-run change in latency introduces unpredictability for continuous operations.

The histogram distributions reinforce this observation. OPC UA's latency values are clustered into several tight peaks within the sub-600 ms range, reflecting its cyclical but controlled variation. MQTT's distribution is bimodal, with latency values concentrated in two distant clusters, highlighting its phase-based behavior but also its significantly higher delay compared to OPC UA.

In summary, OPC UA delivers faster and more predictable feedback performance, making it better suited for high-frequency control loops and real-time applications. MQTT, despite its stability within each phase, operates with much higher average latency, making it more suitable for slower, non-time-critical feedback tasks.

Appendix References

Only the primary model files directly related to the experimental activity are included in the appendix:

- Appendix 1.21: MQTT Pose Zip file – (10 Pairs MQTT pose data)
- Appendix 1.22: OPCUA Pose Zip file – (10 Pairs OPCUA pose data)

4.4. Jitter Analysis:

The amount of variance in the latency between consecutive messages in a data stream is referred to as "jitter" in this study. Jitter quantifies the irregularity of these delays over time, while latency measures the average time delay for a single message to travel from sender to recipient.

Because it impacts the stability and predictability of system responses, jitter is a crucial performance issue in the context of industrial automation. Large variations between messages can contribute to instability in closed-loop control systems, resulting in operational dangers or inefficiencies, even if the average latency is acceptable. For instance, if the incoming position updates come in irregularly, a robotic arm that follows a conveyor belt may begin to oscillate or miss portions.

This metric does not concentrate on absolute communication speed; rather, it assesses short-term timing stability. It is feasible to ascertain whether a protocol is fast and whether it is consistently fast by separating jitter from average latency.

4.4.1 Relevance to This Research:

Jitter is a crucial factor in determining whether a communication protocol is appropriate for time-sensitive industrial applications, claims this thesis. The following succinctly describes its significance:

- Stability of real-time control: Low jitter guarantees that updates to actuators and control loops are spread evenly, preventing undesired oscillations or "stuttering" in system behaviour.

- Predictable automation sequences: Predictability is crucial for coordinating many moving systems in applications like packing lines, CNC machining, or AGV fleet management.

- Safety compliance: To guarantee that emergency stops or interlocks are reliably activated, many industrial standards (such as IEC 61508 for functional safety) need on consistent temporal behaviour.

- Digital twin fidelity: To faithfully replicate the real-world process, a simulation or monitoring system needs to be updated at regular intervals.

- Evaluation of protocol robustness: This statistic is just as crucial as absolute delay since significant jitter can impair system performance even with low latency.

Deterministic control activities are typically better suited for a protocol that produces low and stable jitter, particularly in regulated and safety-critical industries like aircraft, food production, and pharmaceuticals.

4.4.2 Measurement Methodology:

The following measuring protocol was used to provide an equitable and repeatable comparison between OPC UA and MQTT:

- Timestamp Capture ROS 2 Side: The ROS 2 logging node timestamped each incoming pose feedback message (from /box_pose) when it was received. To guarantee precise inter-device timing comparison, NTP was used to synchronize the system clocks on each participating device.
- Calculation of Consecutive Message Interval: The time interval between the arrival of two consecutive messages was determined for every run:

$$\text{interval} = t(i) - t(i-1)$$

Equation 2: Interval Calculation for Jitter

where "t" is the receive time of message "i".

- **Jitter Calculation:**
  Jitter for each interval was calculated as the absolute deviation from the average interval for that run:

$$\text{jitter} = \text{interval} - \text{Avg. interval}$$

Equation 3: Interval Calculation for Jitter

Table 7: Jitter Summary for MQTT

Run_ID	Average Jitter m/s	Max jitter m/s
0	11.917547	1169.626
1	13.568149	1082.674
2	25.802405	2245.951
3	10.010946	1004.609
4	3.259851	22.331
5	2.569804	20.494
6	9.317500	999.654
7	9.912236	1044.034
8	10.640649	1044.034
9	2.247527	11.040

- Overall Average Jitter: 9.925 ms
- Max Jitter Across Runs: 2245.951 ms (Run 3)

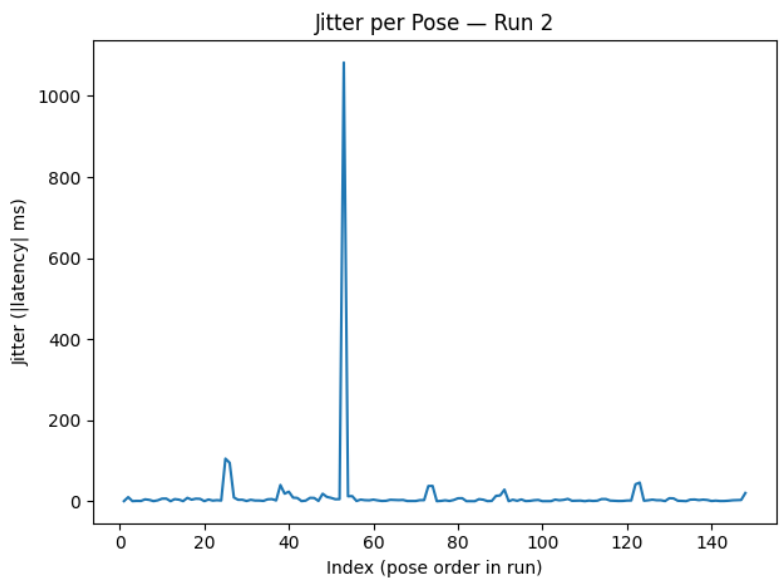

Figure 24: Jitter per Pose – MQTT

MQTT: Most jitter values remain extremely low (close to 0–20 ms), but there is one major outlier spike exceeding 1000 ms around pose index ~55. This indicates that while MQTT can achieve very low jitter for most messages, it is more prone to occasional large disruptions in timing.

Table 8: Jitter Summary for OPCUA

Run_ID	Average Jitter m/s	Max jitter m/s
0	160.029718	412.245
1	158.192882	412.406
2	160.596242	405.219
3	160.063624	410.846
4	160.900014	709.617
5	159.047654	410.218
6	159.886631	413.514
7	160.645317	407.732
8	161.385850	411.390
9	161.195136	410.948

- Overall Average Jitter: 160.194 ms
- Max Jitter Across Runs: 709.617 ms (Run 5)

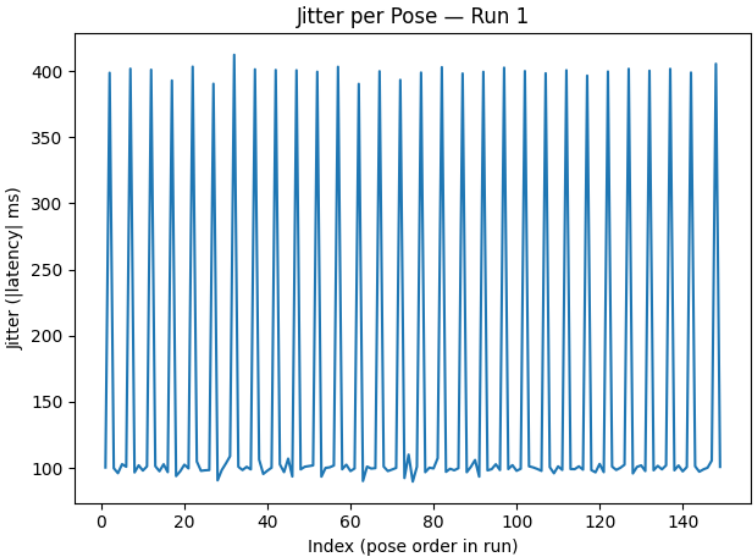

Figure 25: Jitter per Pose – OPCUA

OPC UA: The jitter values show a repetitive oscillation pattern between ~100 ms and ~400 ms, which means the time gap between successive pose feedback is relatively consistent, albeit with a fixed periodic fluctuation. There are no extreme spikes beyond the 400 ms range, indicating controlled and predictable jitter behaviour.

The findings show that OPC UA and MQTT exhibit quite different jitter behaviours.

OPC UA showed a regular oscillatory jitter pattern based on the time-series graphs, with predictable oscillations between around 100 ms and 400 ms over the run. Despite being obvious, this fluctuation is confined and repeats in a regular cycle, suggesting a deterministic and controlled timing profile. Such stability guarantees that, despite the existence of jitter, its frequency and amplitude are predictable, which is a crucial characteristic for deterministic industrial control systems.

On the other hand, MQTT showed incredibly minimal jitter for most of the run, frequently staying between 0–20 ms, indicating remarkable consistency in message delivery under typical circumstances. But in the middle of the run, there was a noticeable outlier spike that lasted more than 1000 ms, indicating an abrupt and serious timing error. In closed-loop control scenarios where consistent message intervals are necessary, this surge disrupts the otherwise smooth delivery pattern and may significantly affect time-sensitive processes.

These conclusions are supported by the histogram distributions. Within the 100–400 ms range, OPC UA's jitter measurements create several tiny peaks, showing its cyclical yet constrained nature. A sparse but large high-jitter event breaks apart the overwhelming concentration of values near zero in the MQTT histogram, suggesting the possibility of uncommon but severe deviations.

In conclusion, OPC UA is beneficial for applications that prioritize deterministic timing because it provides constrained and predictable jitter behaviour. MQTT is best suited for systems where occasional timing errors can be accepted without sacrificing performance or safety since, although it typically delivers incredibly low jitter, it also bears the potential of abrupt big spikes.

4.5. Throughput Analysis:

Throughput, as used in this study, is the number of messages per second (msgs/sec) that feedback messages are successfully sent from the ROS 2 environment (Linux/Gazebo side) to the client system (Windows side) over a certain time period. Throughput is a measure of the communication protocol's ability to send data continuously, as opposed to latency, which counts the time it takes for a single message to travel.

The amount of real-time data that can be sent for process monitoring, control, and synchronization is directly related to throughput in industrial automation. While throughput drops could cause outdated or insufficient data to reach the control system, high throughput guarantees that systems can receive and process frequent changes.

Throughput captures the capacity and sustainability of communication, although it does not measure instantaneous timing precision (like jitter). It is feasible to evaluate a protocol's speed and ability to maintain performance during continuous operation by comparing throughput across protocols.

4.4.2 Relevance to This Research:

A key performance metric for assessing a protocol's appropriateness for industrial control and monitoring is throughput. The following briefly describes its significance:

1. Real-time data availability: More frequent updates from higher throughput enable quicker responses from systems like machine vision setups, robotic manipulators, and AGVs.
2. Digital twin accuracy: By avoiding simulation drift, a higher update rate guarantees that the virtual model stays closely matched with the real system.
3. System coordination: High throughput guarantees that various components stay in sync in multi-robot or multi-sensor situations.
4. Scalability: Without compromising update frequency, systems with high throughput can support additional data sources or complex monitoring.

5. Operational safety: Constant high-rate feedback guarantees that safety systems (such as interlocks and emergency stops) instantly obtain the most recent information.

In general, time-sensitive and data-intensive industrial applications are better suited for protocols that continuously maintain high throughput, especially where continuous feedback is necessary for closed-loop control.

4.4.2 Measurement Methodology:

The following procedure was applied to fairly and consistently compare MQTT and OPC UA throughput performance:

1. **Message Counting — ROS 2 Side:**

The ROS 2 logging node counted the total number of feedback messages published on /box_pose during each test run.

ROS-side throughput was calculated as:

$$\text{ROS throughput } \left(\frac{\text{msgs}}{\text{secs}}\right) = \frac{\text{ROS} - \text{side duration (seconds)}}{\text{Number of messages sent}}$$

Equation 4: ROS throughput Calculation

2. **Message Counting — Windows Side:**

On the client side (MQTT subscriber or OPC UA feedback receiver), the number of messages received was counted and divided by the total reception duration to obtain Windows-side throughput:

$$\text{Windows throughput } \left(\frac{\text{msgs}}{\text{secs}}\right) = \frac{\text{Number of messages recieved}}{\text{Windows} - \text{side duration (seconds)}}$$

Equation 5: Windows throughput Calculation

3. **Duration Measurement:**

   ROS and Windows durations were measured independently to account for possible differences in logging start/stop times and to identify potential delays in processing or buffering.

Table 9: Throughput Summary for MQTT

Run_ID	ROS throughput mgs/sec	Windows throughput mgs/sec	ROS duration sec	Windows duration sec
0	9.429211	10.193404	15.801958	14.617295
1	9.373719	10.059376	15.895506	14.812052
2	9.315125	8.748604	15.995492	17.031288
3	10.065552	9.436900	14.802963	15.789083
4	10.066370	10.067296	14.801761	14.800399
5	10.060628	10.065420	14.810209	14.803158
6	10.071445	9.432438	14.794302	15.796552
7	10.067449	9.404034	14.800174	15.844265
8	9.364768	10.086369	14.415733	13.384400
9	10.063540	10.066333	14.805923	14.801815

Table 10: Throughput Summary for OPCUA

Run_ID	ROS throughput mgs/sec	Windows throughput mgs/sec	ROS duration sec	Windows duration sec
0	10.064856	9.969387	14.903343	15.046061
1	10.068902	10.321067	14.500091	14.048935
2	10.066353	9.963257	14.901127	15.055317
3	10.071869	10.104336	14.098674	14.053373
4	8.888848	10.233134	17.100078	14.560544
5	10.073705	10.203075	12.706348	12.545238
6	10.067294	9.951076	14.899733	15.073747
7	10.081365	9.886251	12.299922	12.542672
8	10.068672	9.827591	14.699058	15.059642
9	10.076153	9.819964	14.688146	15.071339

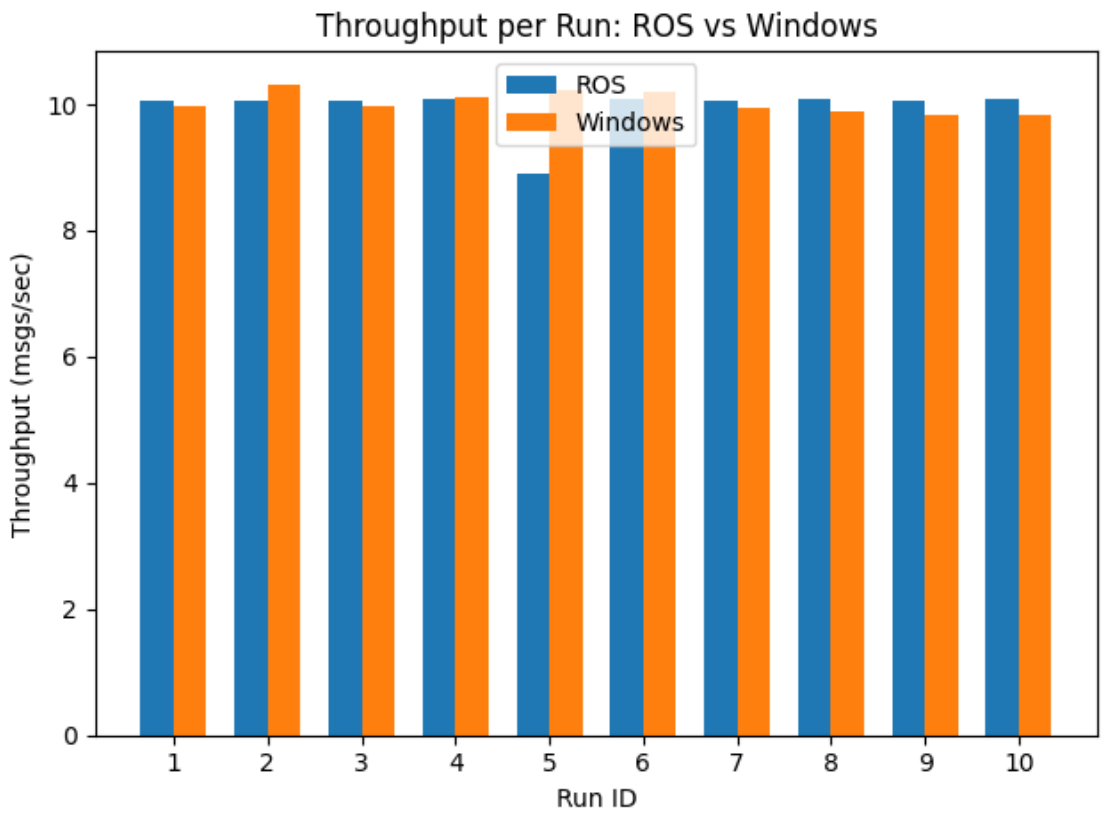

Figure 26: Throughput per run – OPCUA

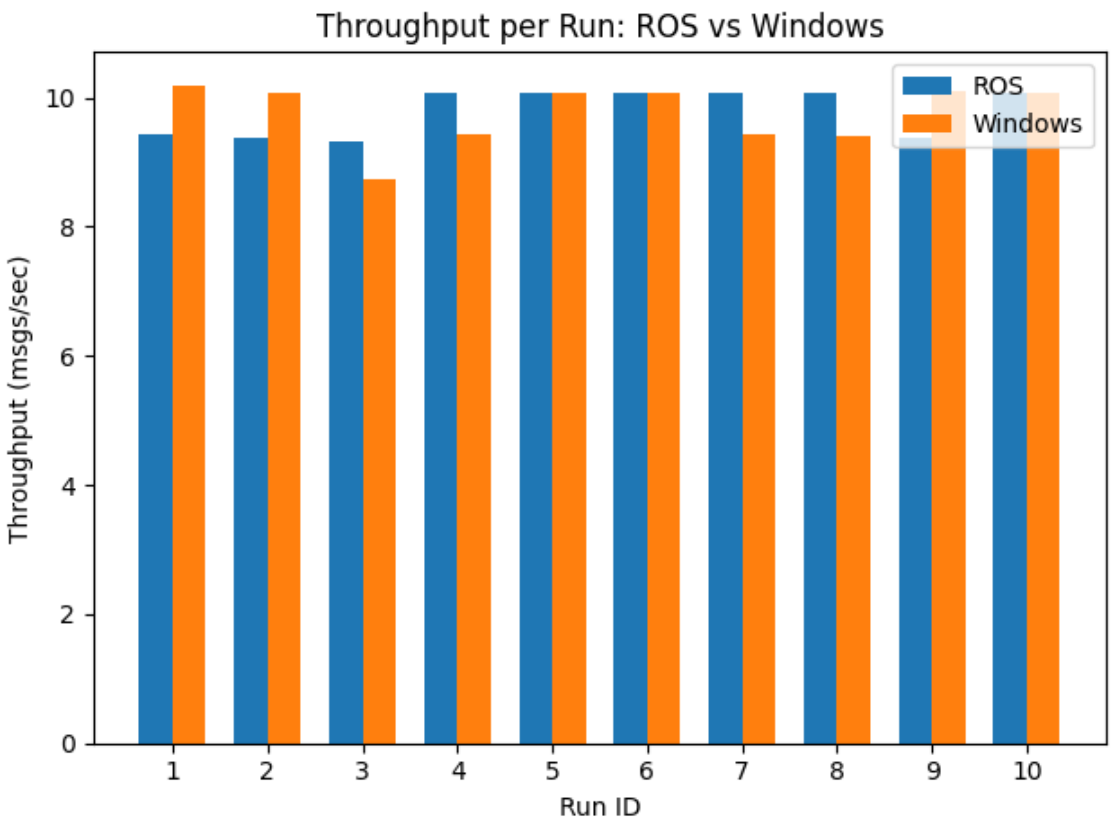

Figure 27: Throughput per run – MQTT

Observation from Time-Series Plots (ROS vs Windows Throughput per Run)

The throughput per run charts for both OPC UA and MQTT display how many messages per second were processed on the ROS side (Linux/Gazebo) versus the Windows client side.

- **OPC UA:**
  - Throughput values on both sides are consistently close to 10 msgs/sec across most runs, with only minor deviations.
  - Differences between ROS and Windows throughput are generally small (±0.3 msgs/sec), except for Run 5, where ROS throughput drops to around 8.9 msgs/sec while Windows maintains over 10 msgs/sec. This outlier suggests a possible short-lived network or processing bottleneck on the ROS side during that run.
  - The consistency between the two sides indicates strong synchronization and minimal message loss in the OPC UA pipeline.

- **MQTT:**
  - Throughput values are more variable compared to OPC UA, with some runs showing noticeable gaps between ROS and Windows throughput.
  - For example, Run 1 and Run 2 have Windows throughput around 0.7–0.8 msgs/sec higher than ROS, suggesting possible buffering or packet handling delays on the ROS side.
  - Conversely, Runs 3, 4, 7, and 8 show negative differences (Windows throughput lower than ROS), which could indicate that the Windows client was unable to process all incoming messages in time, possibly due to spikes in message arrival rates.

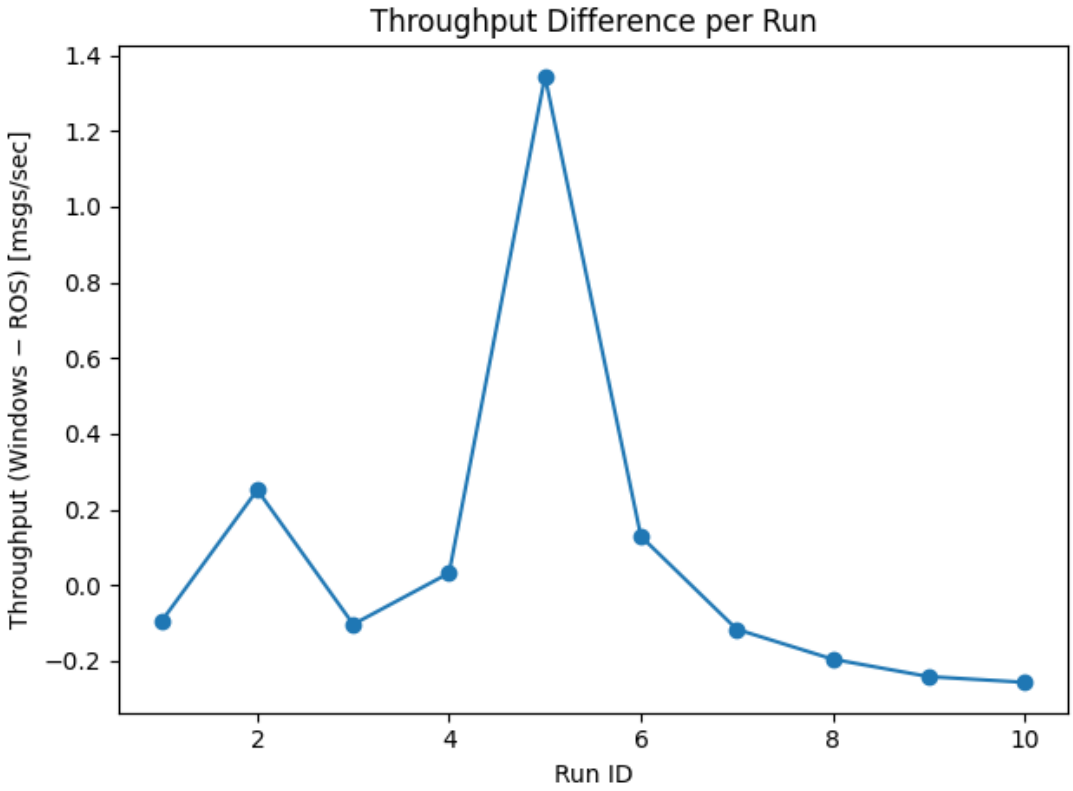

Figure 28: Throughput Differences per run – OPCUA

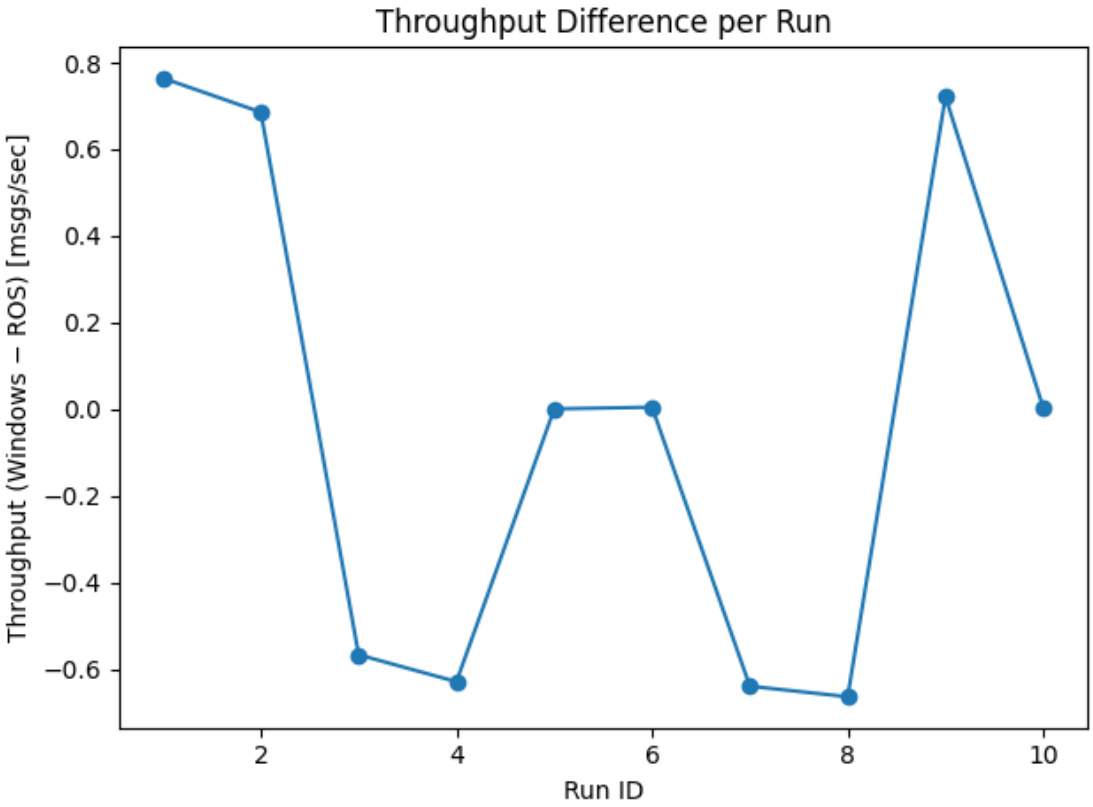

Figure 29: Throughput Differences per run – MQTT

Observation from Difference Plots (Windows – ROS Throughput)

- **OPC UA:**
  - Differences hover close to zero for most runs, showing tight synchronization and balanced processing capacity between sender and receiver.
  - The largest deviation occurs in Run 5 (+1.35 msgs/sec), reinforcing the anomaly noted in the throughput chart.
  - The low variance in differences reflects high protocol reliability for steady, predictable throughput.
- **MQTT:**
  - The difference curve is more volatile, alternating between positive and negative deviations up to ±0.7 msgs/sec.
  - The lack of a consistent bias (positive or negative) indicates that the throughput imbalance shifts direction across runs, possibly due to non-deterministic network packet scheduling or message batching inherent to MQTT's broker-based delivery model.
  - This variability, while moderate, can cause irregular update rates in time-sensitive applications

Overall Outcome

- **OPC UA** delivers **more stable and predictable throughput** across runs, with fewer large deviations between ROS and Windows processing rates.
- **MQTT** achieves throughput close to OPC UA on average, but exhibits **higher variability and occasional imbalances** between sender and receiver, which may cause uneven update pacing in continuous data streams.

Appendix References

Only the primary model files directly related to the experimental activity are included in the appendix:

- Appendix 1.23: Trigger Analysis OPCUA.ipynb
- Appendix 1.24: Pose Analysis MQTT.ipynb
- Appendix 1.25: Trigger Analysis MQTT.ipynb
- Appendix 1.26: Pose Analysis OPCUA .ipynb

5. DISCUSSION OF FINDINGS

This study evaluated the performance of OPC UA and MQTT in terms of data loss, trigger-to-ROS latency, pose-to-feedback latency, jitter, and throughput using a ROS 2-Gazebo-Windows manufacturing simulation.

The findings repeatedly demonstrated that MQTT had greater variability, sporadic spikes, and phase shifts in latency, while OPC UA provided more consistent and reliable performance across the majority of measures.
MQTT's larger variance in timing-related measurements suggests a less predictable communication pattern, even though it frequently reached equivalent average throughput.

Section-wise Discussion:

1. **Data Loss:**

- **Findings:** Across all test runs, OPC UA and MQTT showed no packet loss, indicating both protocols can deliver reliable communication under the simulated load.
- **Interpretation:** From a purely delivery-success perspective, both are viable for industrial messaging.
- **Implication:** The absence of loss in both cases aligns with literature findings on reliability when using TCP-based transports for both protocols.

2. **Trigger-to-ROS Latency:**

- **Findings:** OPC UA maintained latency mostly within 200–1000 ms, with tight clustering in histograms, while MQTT’s latency ranged widely, with values from ~200 ms up to 2.7 seconds and occasional phase jumps.
- **Interpretation:** OPC UA offers better determinism and faster average reaction times for triggers, which is essential in real-time manufacturing control. MQTT’s variable delays could hinder strict cycle-time applications.

- **Comparison to Literature:** Prior studies (Stefan Profanter, 2019) have noted that OPC UA's synchronous request/response and binary encoding reduce latency variance compared to MQTT's publish/subscribe model with broker queuing. Our results confirm this behaviour.

3. **Pose-to-Feedback Latency**

- **Findings:** OPC UA consistently kept feedback latency between 100–550 ms in a cyclical but bounded range, while MQTT operated with two distinct latency plateaus (~2.45 s and ~1.38 s), showing abrupt mid-run shifts.
- **Interpretation:** OPC UA's bounded oscillations make it predictable for continuous feedback loops, whereas MQTT's phase-based latency, though stable within phases, is much higher in absolute terms and can affect real-time responsiveness.
- **Practical Impact:** This supports using OPC UA for applications like robotic alignment or AGV navigation, while MQTT might be suited for slower monitoring tasks.

4. **Jitter**

- **Findings:** OPC UA's jitter was very stable (~160 ms average, small deviation), except for one outlier in Run 5 (709 ms). MQTT's average jitter was far lower (~10 ms), but with extreme spikes in some runs (up to 2.24 seconds).
- **Interpretation:** Low average jitter for MQTT looks favourable, but the presence of large spikes undermines timing predictability. OPC UA's constant cyclic jitter is more acceptable in deterministic control because it's bounded.
- **Industry View:** In safety-critical systems, bounded jitter is often more valuable than a low average with unpredictable outliers.

5. **Throughput**

- **Findings:** Both protocols achieved ~10 msgs/sec throughput on average. OPC UA showed minimal difference between ROS and Windows ends, except for a single run anomaly (+1.35 msgs/sec difference). MQTT displayed alternating positive/negative differences across runs, indicating less consistency.
- **Interpretation:** Both can sustain similar throughput in steady-state conditions, but OPC UA demonstrates tighter sender–receiver synchronization.
- **Implication:** In systems where precise pacing of messages is needed, OPC UA's more consistent throughput may reduce the need for additional buffering or flow control logic.

Methodological Limitations

- The study was conducted in a simulated environment (Gazebo + ROS 2) with a single Windows client, which may not capture all real-world industrial network conditions such as wireless interference, multi-client contention, or high-bandwidth mixed traffic.
- The message payload used in tests was fixed in size; varying payload sizes could alter performance, particularly for MQTT where broker buffering may come into play.
- Testing was limited to **one implementation** of each protocol (open-source OPC UA stack, Eclipse Mosquitto broker). Results may vary with different software/hardware configurations.

6. CONCLUSION

This research systematically evaluated **OPC UA** and **MQTT** in a simulated factory automation environment using ROS 2 and Gazebo, focusing on five key performance metrics:

1. **Data Loss** – Both protocols demonstrated zero data loss under the tested conditions, indicating high reliability in a controlled low-loss LAN environment.
2. **Trigger-to-ROS Latency** – OPC UA consistently delivered lower and more stable latency values, while MQTT exhibited higher variability and occasional peaks, potentially problematic for time-critical tasks.
3. **Pose-to-Feedback Latency** – OPC UA maintained predictable sub-second response times, whereas MQTT operated with significantly higher average latency and sudden phase shifts between latency plateaus.
4. **Jitter** – OPC UA showed uniform jitter behaviour with small variations, supporting deterministic control; MQTT's jitter was generally low but prone to occasional extreme spikes.
5. **Throughput** – No significant throughput advantage was found for MQTT under our low-loss TCP-based environment, in contrast to some literature reports.

These results confirm that OPC UA offers superior consistency and predictability in latency-sensitive industrial control loops, whereas MQTT's performance variability may be acceptable in less time-critical monitoring applications.

Impact on Industry and Society

- **Industrial Automation** – The findings support the adoption of OPC UA for deterministic and safety-critical manufacturing environments, such as pharmaceutical production, automotive assembly, and robotics integration. Its predictability can reduce downtime and improve synchronization between physical and digital systems.

- **IIoT Integration** – MQTT remains a strong candidate for wide-area, cloud-connected monitoring systems where lightweight messaging and broker-based distribution are priorities over strict latency.
- **Digital Twin Development** – OPC UA's stable feedback performance can enhance real-time simulation fidelity, leading to more accurate predictive maintenance and process optimization models.
- **Cross-Domain Relevance** – The results have implications beyond manufacturing, including autonomous vehicle coordination, energy grid control, and healthcare device monitoring, where latency consistency is paramount.

Lessons Learned

- **Controlled Environment Effects** – The lack of data loss in both protocols reflects the stability of the testbed's LAN setup; results may differ in noisy wireless or high-loss networks.
- **Protocol Tuning Matters** – The performance gaps could potentially narrow or widen depending on QoS settings, broker/server placement, and encoding options.
- **Measurement Synchronization** – NTP clock alignment across devices was essential for valid cross-platform latency comparison. This step is often overlooked in less rigorous protocol benchmarking.
- **Literature Alignment and Divergence** – While our latency and jitter results align with studies highlighting OPC UA's deterministic performance, our throughput findings diverge from works that show MQTT superiority, emphasizing the importance of context-specific testing.

Potential for Further Research

Future research can extend this work in several promising directions:

1. **WAN and High-Loss Network Testing** – Introduce packet loss, network congestion, and variable bandwidth to assess robustness under realistic industrial IoT conditions.

2. **Hybrid Protocol Architectures** – Explore scenarios where OPC UA is used for control loops and MQTT for high-volume monitoring, integrating both into a unified control plane.
3. **Security Impact** – Evaluate how encryption (TLS/SSL) and authentication mechanisms affect performance metrics for each protocol.
4. **Scalability Analysis** – Test with a higher number of simultaneous devices and message topics to assess multi-node scaling.
5. **QoS Level Tuning** – Systematically vary MQTT QoS levels and OPC UA publish intervals to identify optimal configurations for different use cases.

Take-Home Messages

- For time-sensitive control, OPC UA consistently outperforms MQTT in latency predictability and jitter stability.
- For monitoring and telemetry, MQTT remains a viable choice, especially when message overhead and broker-based scalability are priorities.
- Protocol selection in industrial automation should be driven not by theoretical throughput claims alone, but by measured performance in the specific operational context.
- The coexistence of both protocols, each in its optimal role, can offer the best of both worlds for Industry 4.0 deployments.

APPENDIX I

- Appendix 1.1: factory.world – The factory world.
- Appendix 1.2: model.sdf – Box model.
- Appendix 1.3: model.sdf – Conveyor model.
- Appendix 1.4: BoxPosePublisher.cc
- Appendix 1.5: BoxMoverPlugin.cc
- Appendix 1.6: BoxMoverPlugin.hh
- Appendix 1.7: gazebo_launch.py
- Appendix 1.8: box_pose_monitor.py
- Appendix 1.9: box_teleport_node.py
- Appendix 1.10: box_motion.launch.py
- Appendix 1.11: opcua_trigger_client.py
- Appendix 1.12: write_opcua_command.py
- Appendix 1.13: opcua_feedback_server.py
- Appendix 1.14: MQTT_trigger_listener.py
- Appendix 1.15: MQTT Subscribe.py
- Appendix 1.16: MQTT Publish.py
- Appendix 1.17: TimeCheck.py
- Appendix 1.18: CheckingTime.py
- Appendix 1.19: MQTT_Trigger Zip File
- Appendix 1.20: OPCUA_Trigger Zip file
- Appendix 1.21: MQTT Pose Zip file – (10 Pairs MQTT pose data)
- Appendix 1.22: OPCUA Pose Zip file – (10 Pairs OPCUA pose data)
- Appendix 1.23: Trigger Analysis OPCUA.ipynb
- Appendix 1.24: Pose Analysis MQTT.ipynb
- Appendix 1.25: Trigger Analysis MQTT.ipynb
- Appendix 1.26: Pose Analysis OPCUA .ipynb

ALPHABETICAL LIST OF REFERENCES

- (Abdulkadir Karaagac, 2019) The Integration of LwM2M and OPC UA: An Interoperability Approach for Industrial IoT, IEEE 5th World Forum on Internet of Things (WF-IoT) https://www.researchgate.net/publication/332572333_The_Integration_of_LwM2M_and_OPC_UA_An_Interoperability_Approach_for_Industrial_IoT

- (Andreas Burger, 2019) Bottleneck Identification and Performance Modeling of OPC UA Communication Models, https://research.spec.org/icpe_proceedings/2019/proceedings/p231.pdf

- (Aparajita Tripathy, 2022) Interoperability between ROS and OPC UA: A local cloud-based approach. IEEE 5th International Conference on Industrial Cyber-Physical Systems (ICPS) https://www.diva-portal.org/smash/get/diva2%3A1664464/FULLTEXT01.pdf

- APMonitor. OPC transfer documentation. https://apmonitor.com/dde/index.php/Main/OPCTransfer

- (Banno, 2023) Performance evaluation of MQTT communication with heterogeneous traffic. IEEE COMPSAC, https://www.rbanno.net/data/paper/202306_IEEE_COMPSAC.pdf

- (ChatGPT, 2023) Large language model, from https://chat.openai.com/

- (Cüneyt Bayılmıs, 2022) A survey on communication protocols and performance evaluations for Internet of Things. Digital Communications and Networks, https://www.sciencedirect.com/science/article/pii/S2352864822000347

- (Daniel Silva, 2021) A Performance Analysis of Internet of Things Networking Protocols: Evaluating MQTT, CoAP, OPC UA, MDPI https://www.mdpi.com/2076-3417/11/11/4879

- Eclipse Foundation paho.mqtt.client Python source code https://github.com/eclipse/paho.mqtt.python/blob/master/src/paho/mqtt/client.py

- (Endre Erős, 2019) A ROS2 based communication architecture for control in collaborative and intelligent automation systems, ELSEVIER https://www.sciencedirect.com/science/article/pii/S2351978920300469

- FreeOpcUa. *FreeOpcUa GitHub repository*. https://github.com/FreeOpcUa

- FreeOpcUa. *Python OPC UA server source code*. https://github.com/FreeOpcUa/python-opcua/blob/master/opcua/server/server.py

- HiveMQ. MQTT Essentials – A ten-part blog series. https://www.hivemq.com/mqtt-essentials/

- (Isaías González, 2019) A Literature Survey on Open Platform Communications (OPC) Applied to Advanced Industrial Environments, MDPI https://www.mdpi.com/2079-9292/8/5/510

- (Jaloudi, 2019) Communication Protocols of an Industrial Internet of Things Environment: A Comparative Study, MDPI https://www.mdpi.com/1999-5903/11/3/66

- (L. Freitas, 2024) OPC-UA in interoperability – A performance comparative testing. Procedia Computer Science, https://www.sciencedirect.com/science/article/pii/S2405896324008498

- (Marc Ladegourdie, 2022) Performance Analysis of OPC UA for Industrial Interoperability towards Industry 4.0, MDPI https://www.mdpi.com/2624-831X/3/4/27

- (Martin Dahl, 2019) Sequence Planner - Automated Planning and Control for ROS2-based Collaborative and Intelligent Automation Systems, https://arxiv.org/abs/1903.05850

- (Matteo Pernoa, 2021) Implementation of digital twins in the process industry: A systematic literature review of enablers and barriers, ELSEVIER https://www.sciencedirect.com/science/article/pii/S0166361521001652?utm

- Python OPC UA. (n.d.). Server documentation. Retrieved https://python-opcua.readthedocs.io/en/latest/server.html

- (Salvatore Cavalieri, 2013) Analysis of OPC UA performances, ELSEVIER https://www.sciencedirect.com/science/article/abs/pii/S0920548913000640

- (Santhana Pandiyan Muniraj, 2021) An implementation of OPC UA for machine-to-machine communications in a smart factory. Procedia Manufacturing. https://www.sciencedirect.com/science/article/pii/S2351978921000093

- (Seung-Yong Lee, 2022) OPC-UA Agent for Legacy Programmable Logic Controllers, ResearchGate https://www.researchgate.net/publication/363304148_OPC-UA_Agent_for_Legacy_Programmable_Logic_Controllers

- (Tobias Kronauer, 2021) Latency Analysis of ROS2 Multi-Node Systems https://arxiv.org/abs/2101.02074

- (Usmani, 2021) MQTT Protocol for the IoT - Review Paper, ResearchGate https://www.researchgate.net/publication/373640610_MQTT_Protocol_for_the_IoT_-_Review_Paper

CHRONOLOGICAL LIST OF REFERENCES

- (Salvatore Cavalieri, 2013) Analysis of OPC UA performances, ELSEVIER https://www.sciencedirect.com/science/article/abs/pii/S0920548913000640

- (Martin Dahl, 2019) Sequence Planner - Automated Planning and Control for ROS2-based Collaborative and Intelligent Automation Systems, https://arxiv.org/abs/1903.05850

- (Endre Erős, 2019) A ROS2 based communication architecture for control in collaborative and intelligent automation systems, ELSEVIER https://www.sciencedirect.com/science/article/pii/S2351978920300469

- (Abdulkadir Karaagac, 2019) The Integration of LwM2M and OPC UA: An Interoperability Approach for Industrial IoT, IEEE 5th World Forum on Internet of Things (WF-IoT) https://www.researchgate.net/publication/332572333_The_Integration_of_LwM2M_and_OPC_UA_An_Interoperability_Approach_for_Industrial_IoT

- (Andreas Burger, 2019) Bottleneck Identification and Performance Modeling of OPC UA Communication Models, https://research.spec.org/icpe_proceedings/2019/proceedings/p231.pdf

- (Isaías González, 2019) A Literature Survey on Open Platform Communications (OPC) Applied to Advanced Industrial Environments, MDPI https://www.mdpi.com/2079-9292/8/5/510

- (Jaloudi, 2019) Communication Protocols of an Industrial Internet of Things Environment: A Comparative Study, MDPI https://www.mdpi.com/1999-5903/11/3/66

- (Santhana Pandiyan Muniraj, 2021) An implementation of OPC UA for machine-to-machine communications in a smart factory. Procedia Manufacturing. https://www.sciencedirect.com/science/article/pii/S2351978921000093

- (Daniel Silva, 2021) A Performance Analysis of Internet of Things Networking Protocols: Evaluating MQTT, CoAP, OPC UA, MDPI https://www.mdpi.com/2076-3417/11/11/4879

- (Matteo Pernoa, 2021) Implementation of digital twins in the process industry: A systematic literature review of enablers and barriers, ELSEVIER https://www.sciencedirect.com/science/article/pii/S0166361521001652?utm

- (Tobias Kronauer, 2021) Latency Analysis of ROS2 Multi-Node Systems https://arxiv.org/abs/2101.02074

- (Usmani, 2021) MQTT Protocol for the IoT - Review Paper, ResearchGate https://www.researchgate.net/publication/373640610_MQTT_Protocol_for_the_IoT_-_Review_Paper

- (Seung-Yong Lee, 2022) OPC-UA Agent for Legacy Programmable Logic Controllers, ResearchGate https://www.researchgate.net/publication/363304148_OPC-UA_Agent_for_Legacy_Programmable_Logic_Controllers

- (Cüneyt Bayılmıs, 2022) A survey on communication protocols and performance evaluations for Internet of Things. Digital Communications and Networks, https://www.sciencedirect.com/science/article/pii/S2352864822000347

- (Marc Ladegourdie, 2022) Performance Analysis of OPC UA for Industrial Interoperability towards Industry 4.0, MDPI https://www.mdpi.com/2624-831X/3/4/27

- (Aparajita Tripathy, 2022) Interoperability between ROS and OPC UA: A local cloud-based approach. IEEE 5th International Conference on Industrial Cyber-Physical Systems (ICPS) https://www.diva-portal.org/smash/get/diva2%3A1664464/FULLTEXT01.pdf

- (Banno, 2023) Performance evaluation of MQTT communication with heterogeneous traffic. IEEE COMPSAC, https://www.rbanno.net/data/paper/202306_IEEE_COMPSAC.pdf

- (L. Freitas, 2024) OPC-UA in interoperability – A performance comparative testing. Procedia Computer Science, https://www.sciencedirect.com/science/article/pii/S2405896324008498

- APMonitor:. OPC transfer documentation. https://apmonitor.com/dde/index.php/Main/OPCTransfer

- Python OPC UA: Server documentation. https://python-opcua.readthedocs.io/en/latest/server.html

- FreeOpcUa.: FreeOpcUa GitHub repository. https://github.com/FreeOpcUa

- FreeOpcUa.: Python OPC UA server source code https://github.com/FreeOpcUa/python-opcua/blob/master/opcua/server/server.py

- HiveMQ. MQTT Essentials – A ten-part blog series. https://www.hivemq.com/mqtt-essentials/

- Eclipse Foundation paho.mqtt.client Python source code https://github.com/eclipse/paho.mqtt.python/blob/master/src/paho/mqtt/client.py

- (ChatGPT, 2023) Large language model, from https://chat.openai.com/

REFERENCE LIST BY BIBLIOGRAPHY TYPE

Online sources:

1. APMonitor:. OPC transfer documentation. https://apmonitor.com/dde/index.php/Main/OPCTransfer

2. Python OPC UA: Server documentation. https://python-opcua.readthedocs.io/en/latest/server.html

3. FreeOpcUa.: FreeOpcUa GitHub repository. https://github.com/FreeOpcUa

4. FreeOpcUa.: Python OPC UA server source code https://github.com/FreeOpcUa/python-opcua/blob/master/opcua/server/server.py

5. HiveMQ. MQTT Essentials – A ten-part blog series. https://www.hivemq.com/mqtt-essentials/

6. Eclipse Foundation paho.mqtt.client Python source code https://github.com/eclipse/paho.mqtt.python/blob/master/src/paho/mqtt/client.py

7. (ChatGPT, 2023) Large language model, from https://chat.openai.com/

Scientific Articles:

- (Salvatore Cavalieri, 2013) Analysis of OPC UA performances, ELSEVIER https://www.sciencedirect.com/science/article/abs/pii/S0920548913000640

- (Martin Dahl, 2019) Sequence Planner - Automated Planning and Control for ROS2-based Collaborative and Intelligent Automation Systems, https://arxiv.org/abs/1903.05850

- (Endre Erős, 2019) A ROS2 based communication architecture for control in collaborative and intelligent automation systems, ELSEVIER https://www.sciencedirect.com/science/article/pii/S2351978920300469

- (Abdulkadir Karaagac, 2019) The Integration of LwM2M and OPC UA: An Interoperability Approach for Industrial IoT, IEEE 5th World Forum on Internet of Things (WF-IoT) https://www.researchgate.net/publication/332572333_The_Integration_of_LwM2M_and_OPC_UA_An_Interoperability_Approach_for_Industrial_IoT

- (Andreas Burger, 2019) Bottleneck Identification and Performance Modeling of OPC UA Communication Models, https://research.spec.org/icpe_proceedings/2019/proceedings/p231.pdf

- (Isaías González, 2019) A Literature Survey on Open Platform Communications (OPC) Applied to Advanced Industrial Environments, MDPI https://www.mdpi.com/2079-9292/8/5/510

- (Jaloudi, 2019) Communication Protocols of an Industrial Internet of Things Environment: A Comparative Study, MDPI https://www.mdpi.com/1999-5903/11/3/66

- (Santhana Pandiyan Muniraj, 2021) An implementation of OPC UA for machine-to-machine communications in a smart factory. Procedia Manufacturing. https://www.sciencedirect.com/science/article/pii/S2351978921000093

- (Daniel Silva, 2021) A Performance Analysis of Internet of Things Networking Protocols: Evaluating MQTT, CoAP, OPC UA, MDPI https://www.mdpi.com/2076-3417/11/11/4879

- (Matteo Pernoa, 2021) Implementation of digital twins in the process industry: A systematic literature review of enablers and barriers, ELSEVIER https://www.sciencedirect.com/science/article/pii/S0166361521001652?utm

- (Tobias Kronauer, 2021) Latency Analysis of ROS2 Multi-Node Systems https://arxiv.org/abs/2101.02074

- (Usmani, 2021) MQTT Protocol for the IoT - Review Paper, ResearchGate https://www.researchgate.net/publication/373640610_MQTT_Protocol_for_the_IoT_-_Review_Paper

- (Seung-Yong Lee, 2022) OPC-UA Agent for Legacy Programmable Logic Controllers, ResearchGate https://www.researchgate.net/publication/363304148_OPC-UA_Agent_for_Legacy_Programmable_Logic_Controllers

- (Cüneyt Bayılmıs, 2022) A survey on communication protocols and performance evaluations for Internet of Things. Digital Communications and Networks, https://www.sciencedirect.com/science/article/pii/S2352864822000347

- (Marc Ladegourdie, 2022) Performance Analysis of OPC UA for Industrial Interoperability towards Industry 4.0, MDPI https://www.mdpi.com/2624-831X/3/4/27

- (Aparajita Tripathy, 2022) Interoperability between ROS and OPC UA: A local cloud-based approach. IEEE 5th International Conference on Industrial Cyber-Physical Systems (ICPS) https://www.diva-portal.org/smash/get/diva2%3A1664464/FULLTEXT01.pdf

- (Banno, 2023) Performance evaluation of MQTT communication with heterogeneous traffic. IEEE COMPSAC, https://www.rbanno.net/data/paper/202306_IEEE_COMPSAC.pdf

- (L. Freitas, 2024) OPC-UA in interoperability – A performance comparative testing. Procedia Computer Science, https://www.sciencedirect.com/science/article/pii/S2405896324008498

LIST OF GENERATIVE AI TOOL USAGES

In this work, generative artificial intelligence tools were used for the following purpose(s):

Collection of conceptual explications on the topic of this work	☐
Generation of approaches towards scientific methodology or artistic development of this work	☐
Literature search	☐
Literature analysis	☐
Selection of methods and/or models	☐
Generation of code	☒
Generation of visualizations	☐
Generation of musical compositions and/or arrangements	☐
Structuring of the text of this work	☒
Formulation/Editing of the text of this work	☒
Translation of the text of this work	☐
Generation of presentation contents on the basis of this work	☐
Other:	☐

AFFIDAVIT

I hereby declare in lieu of an oath that I have independently written this work titled

Integration and Performance Analysis of OPC UA and MQTT for Decentralized Industrial Control Between PLC and ROS 2 Environments

and all related parts and that the work is in accordance with the version submitted in digital form. Only the sources and aids expressly named in the work have been used. I have marked as such any ideas taken over directly or in substance.

If generative AI tools were used to write the paper, this has been explicitly listed in the addendum at the end of this submission. Within the core text of my work, I have denoted all relevant contents that were created or paraphrased by generative AI (such as text passages, structures, images, and code or parts thereof) according to the citation style I used for short referencing secondary sources in my submission. In my list of references, each use of generative AI has been listed according to the citation style I used for full references, including the name of the tool, its version number, the date of retrieval, and the prompts used.

I hereby declare that I have not submitted this work in identical or similar form at this or another university previously.

Berlin, 25/08/2025	Rohitsurya Sundara Rajan
Location, Date	Name / Signature

Legal Caution

SRH Universities require an affidavit of independent authenticity of submitted scientific works in order to be assured that the signatory has generated the latter by him- or herself.

Since the law attributes a special relevance to affidavits and arising consequences thereof, submission of a false affidavit entails criminal punishment. Deliberately (and therefore knowingly) submitting a false affidavit may be punishable by up to three years imprisonment or a fine. Negligently submitting an affidavit (i.e. submitting despite having been able to recognize that the affidavit does not match factual truths) may be punishable by up to one year imprisonment or a fine.

The punitive frameworks are set forth in Section 156 of the German Penal Code (*falsche Versicherungen an Eides Statt*), as well as in Section 161 of the German Penal Code (*fahrlässiger Falscheid, fahrlässige falsche Versicherung an Eides Statt*).

Section 156 StGB: False declaration in lieu of oath

Whoever falsely makes a declaration in lieu of an oath before an authority which is competent to administer such declarations or falsely testifies whilst referring to such a declaration incurs a penalty of imprisonment for a term not exceeding three years or a fine.

Section 161 StGB: Negligent false oath; negligent false declaration in lieu of oath

(1) Whoever commits one of the offences referred to in sections 154 to 156 by negligence incurs a penalty of imprisonment for a term not exceeding one year or a fine.
(2) No penalty is incurred if the offender corrects the false statement in time. The provisions of section 158 (2) and (3) apply accordingly.

I hereby acknowledge the above

Berlin, 25/08/2025 Rohitsurya Sundara Rajan

Location, Date Name Signature